

Topological Sort

Minimum Spanning Trees

Review: Breadth-First Search

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};           // Q is a queue (duh); initialize to s  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;      v->d represents depth in tree  
                v->p = u;           v->p represents parent in tree  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

Review: DFS Code

```
DFS (G)
```

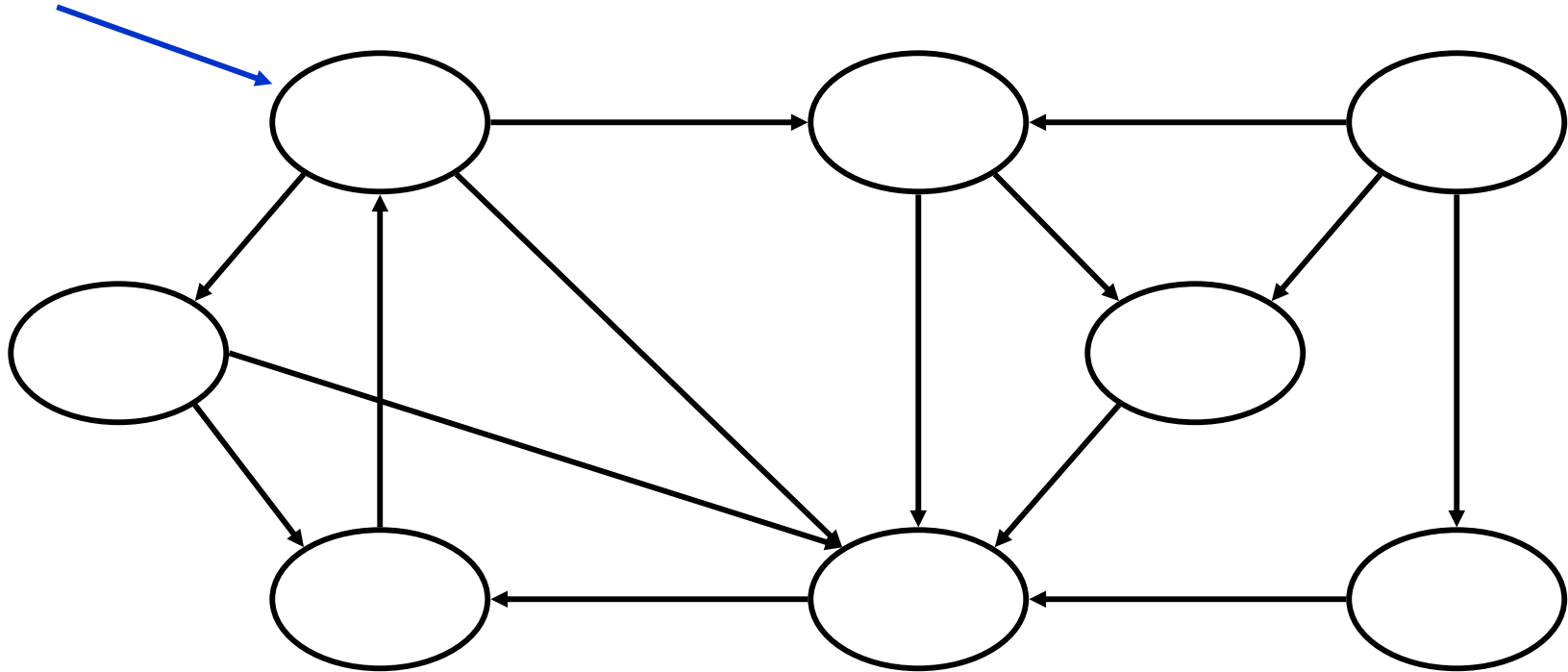
```
{  
    for each vertex  $u \in G \rightarrow V$   
    {  
         $u \rightarrow \text{color} = \text{WHITE};$   
    }  
    time = 0;  
    for each vertex  $u \in G \rightarrow V$   
    {  
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )  
            DFS_Visit( $u$ );  
    }  
}
```

```
DFS_Visit( $u$ )
```

```
{  
     $u \rightarrow \text{color} = \text{YELLOW};$   
    time = time+1;  
     $u \rightarrow d = \text{time};$   
    for each  $v \in u \rightarrow \text{Adj}[]$   
    {  
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )  
            DFS_Visit( $v$ );  
    }  
     $u \rightarrow \text{color} = \text{BLACK};$   
    time = time+1;  
     $u \rightarrow f = \text{time};$   
}
```

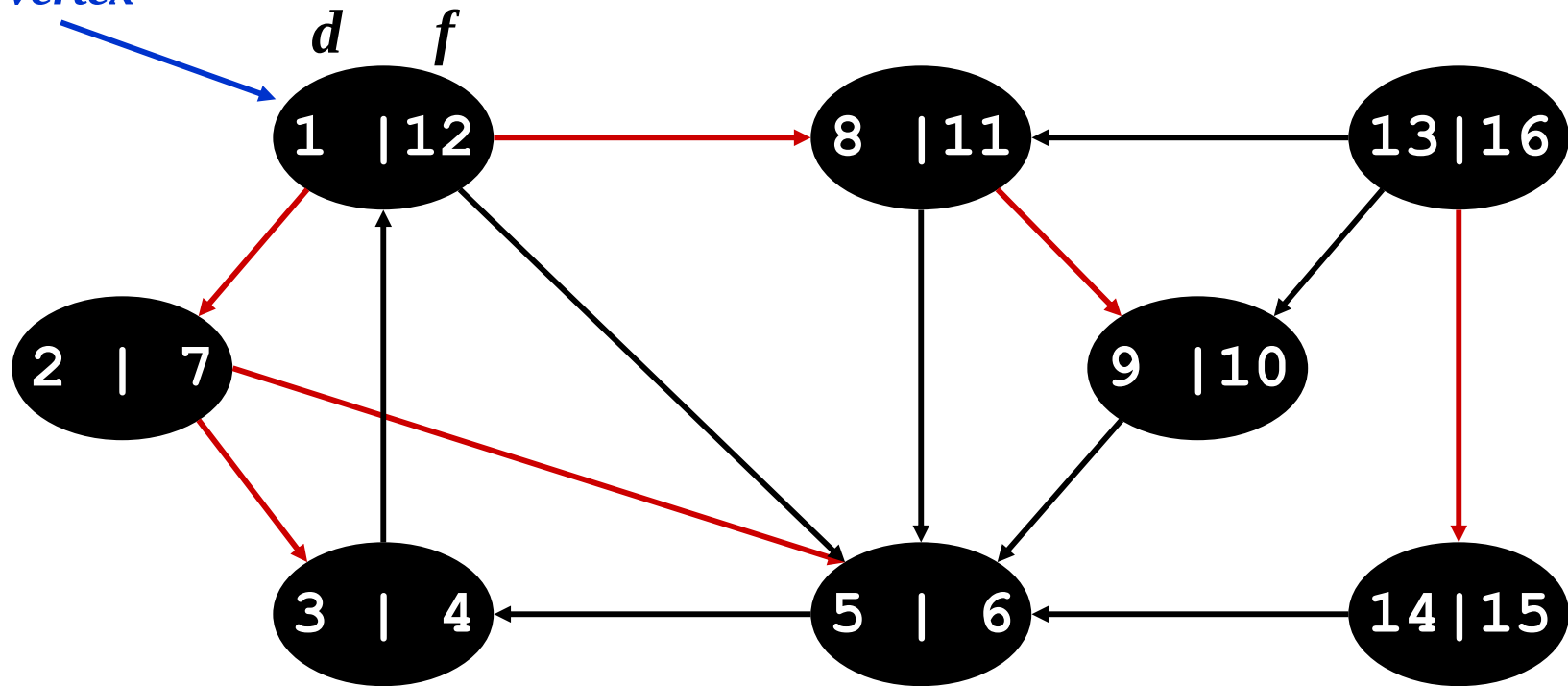
Review: DFS Example

*source
vertex*



Review: DFS Example

source
vertex

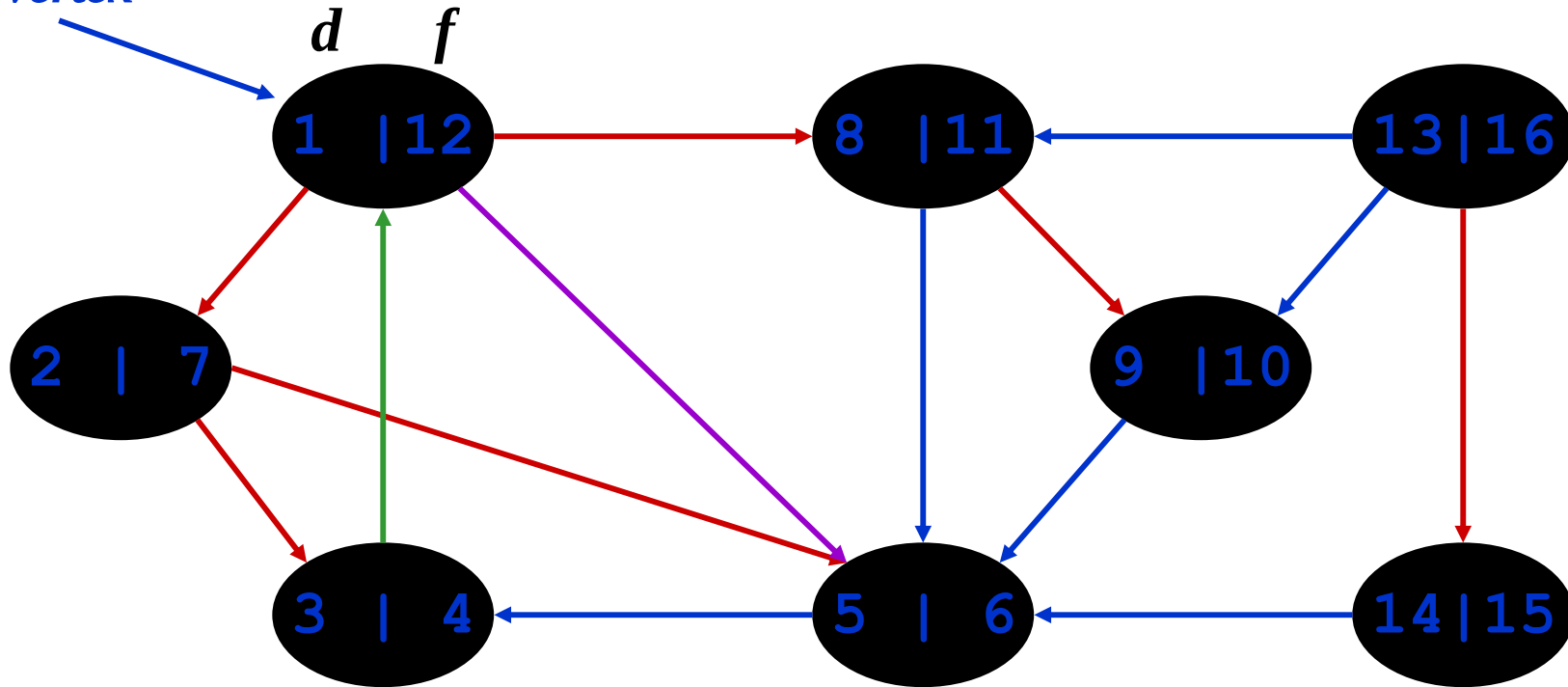


Review: Kinds Of Edges

- Thm: If G is undirected, a DFS produces only tree and back edges
- Thm: An undirected graph is *acyclic* iff a DFS yields no back edges
- Thus, can run DFS to find cycles

Review: Kinds of Edges

source
vertex



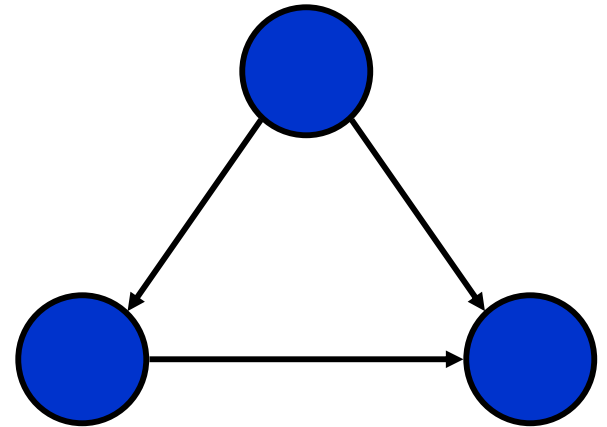
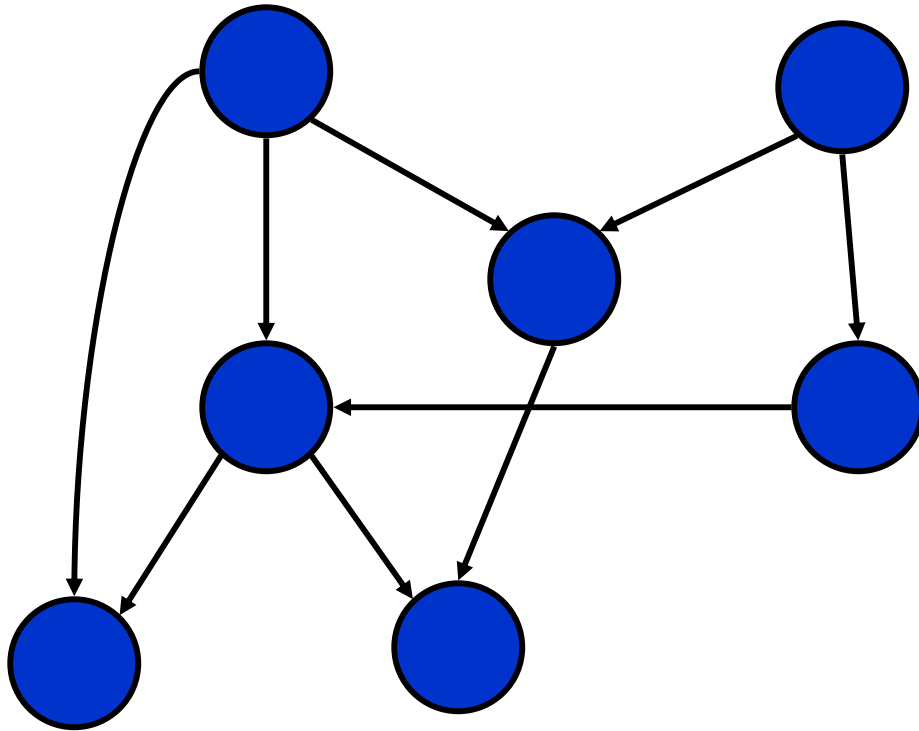
Tree edges *Back edges* *Forward edges* *Cross edges*

DFS And Cycles

- Running time: $O(V+E)$
- We can actually determine if cycles exist in $O(V)$ time:
 - In an undirected acyclic forest, $|E| \leq |V| - 1$
 - So count the edges: if ever see $|V|$ distinct edges, must have seen a back edge along the way
 - *Why not just test if $|E| < |V|$ and answer the question in constant time?*

Directed Acyclic Graphs

- A *directed acyclic graph* or *DAG* is a directed graph with no directed cycles:



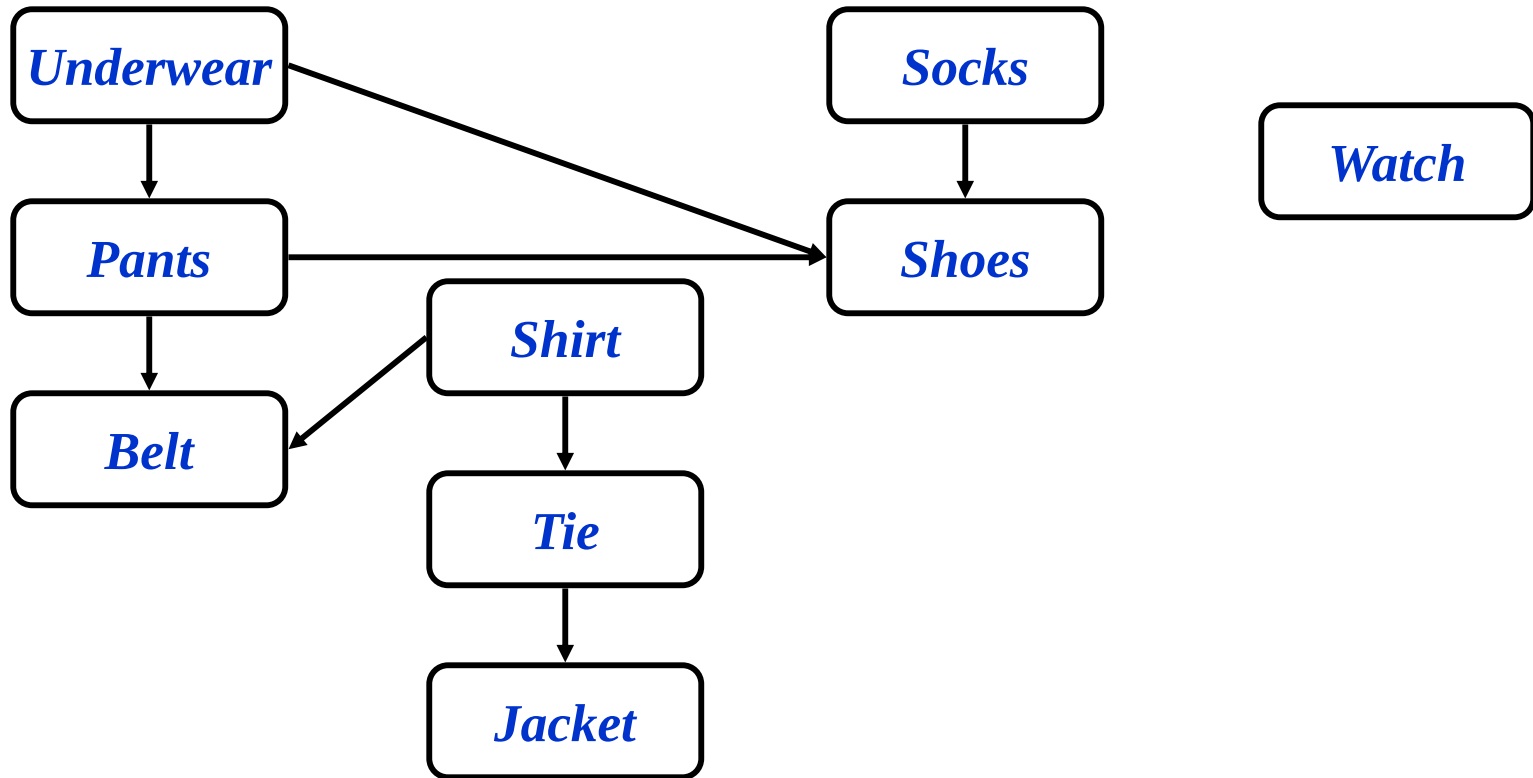
DFS and DAGs

- Argue that a directed graph G is acyclic iff a DFS of G yields no back edges:
 - Forward: if G is acyclic, will be no back edges
 - Trivial: a back edge implies a cycle
 - Backward: if no back edges, G is acyclic
 - Argue contrapositive: G has a cycle $\Rightarrow \exists$ a back edge
 - ◆ Let v be the vertex on the cycle first discovered, and u be the predecessor of v on the cycle
 - ◆ When v discovered, whole cycle is white
 - ◆ Must visit everything reachable from v before returning from DFS-Visit()
 - ◆ So path from $u \rightarrow v$ is yellow $\rightarrow$ yellow, thus (u, v) is a back edge

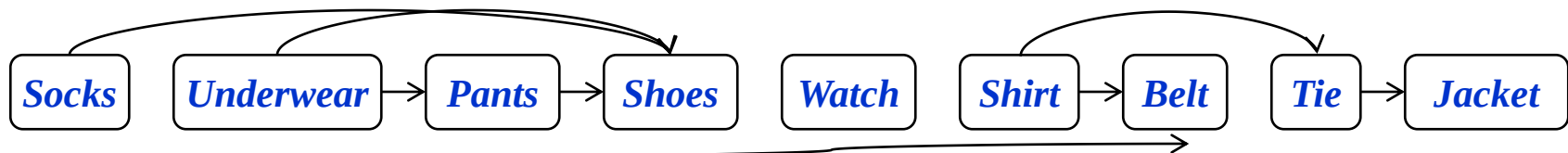
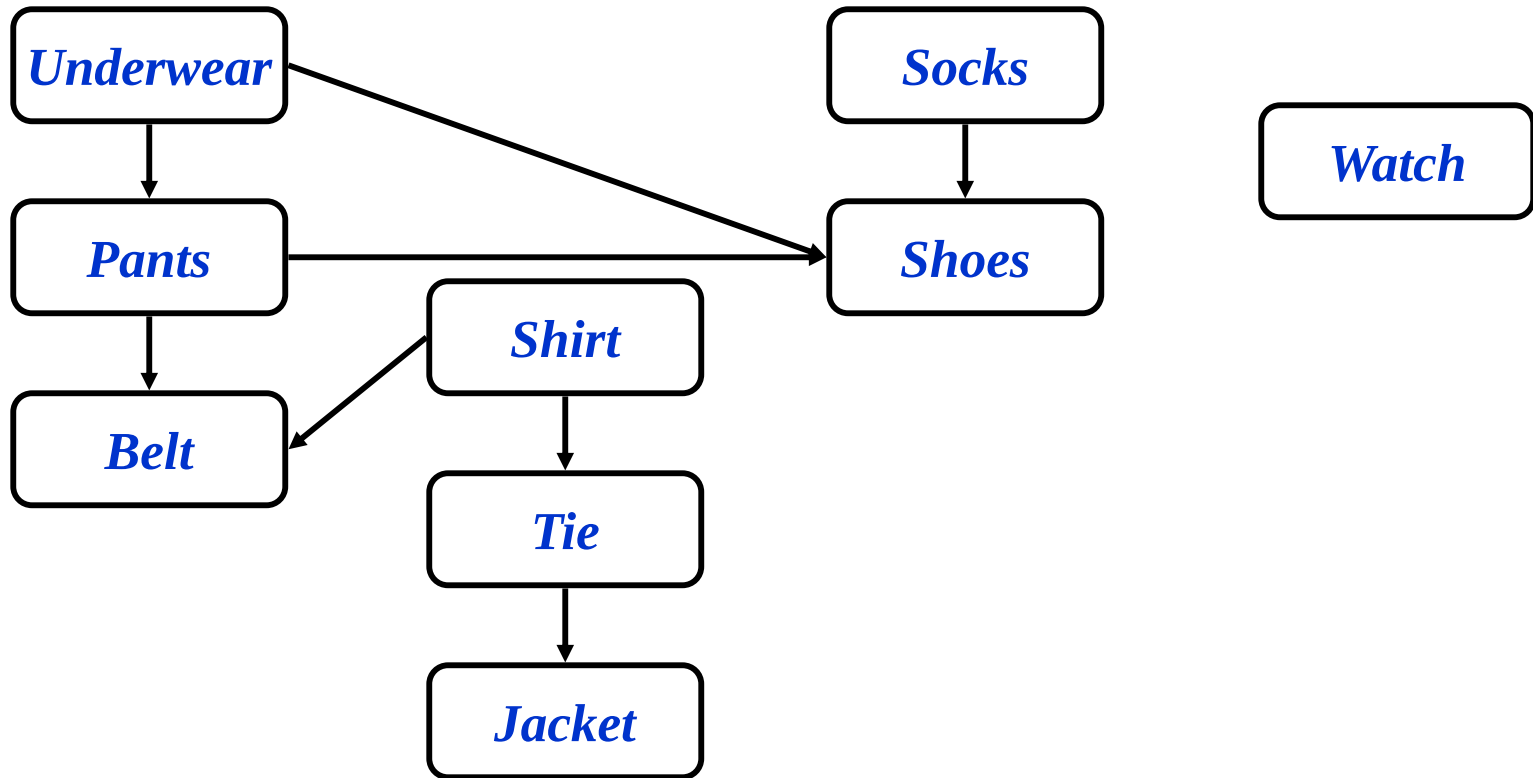
Topological Sort

- *Topological sort* of a DAG:
 - Linear ordering of all vertices in graph G such that vertex u comes before vertex v if edge $(u, v) \in G$
- Real-world example: getting dressed

Getting Dressed



Getting Dressed



Topological Sort Algorithm

```
Topological-Sort()
```

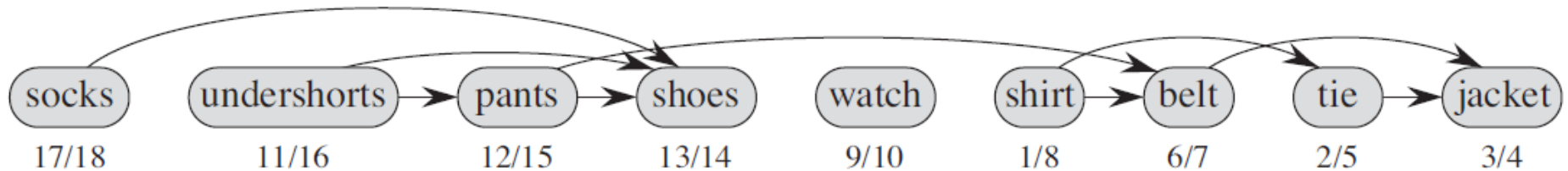
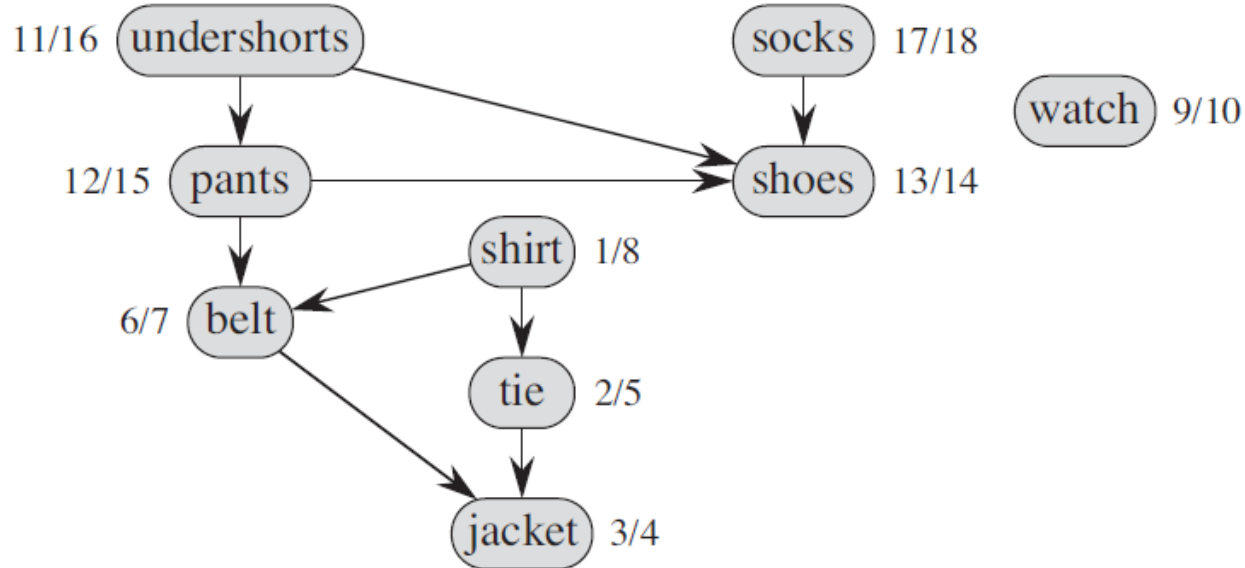
```
{  
    Run DFS  
    When a vertex is finished, output it  
    Vertices are output in reverse  
    topological order  
}
```

- Time: $O(V+E)$
- Correctness: Want to prove that
$$(u,v) \in G \Rightarrow u \rightarrow f > v \rightarrow f$$

Correctness of Topological Sort

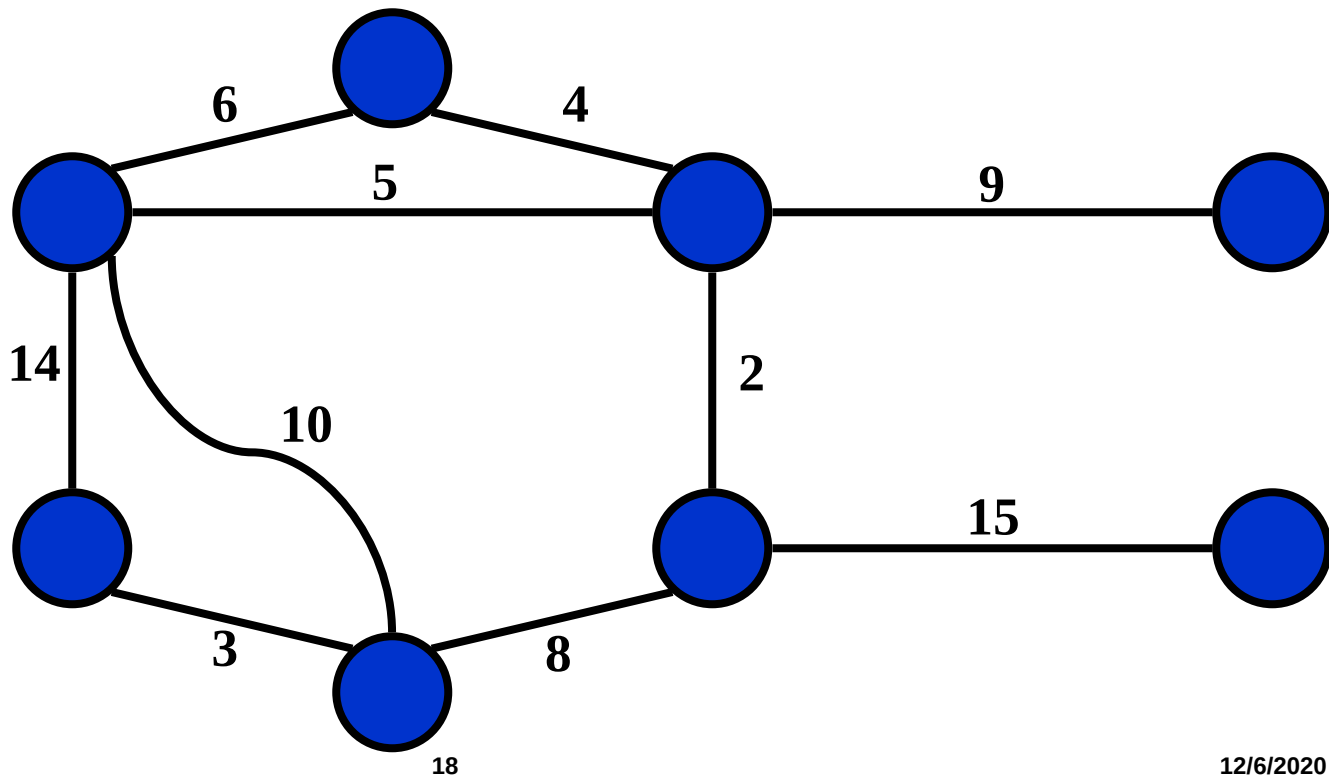
- Claim: $(u,v) \in G \Rightarrow u \rightarrow f > v \rightarrow f$
 - When (u,v) is explored, u is yellow
 - $v = \text{yellow} \Rightarrow (u,v)$ is back edge. Contradiction (*Why?*)
 - $v = \text{white} \Rightarrow v$ becomes descendent of $u \Rightarrow v \rightarrow f < u \rightarrow f$
(since must finish v before backtracking and finishing u)
 - $v = \text{black} \Rightarrow v$ already finished $\Rightarrow v \rightarrow f < u \rightarrow f$

Getting Dressed



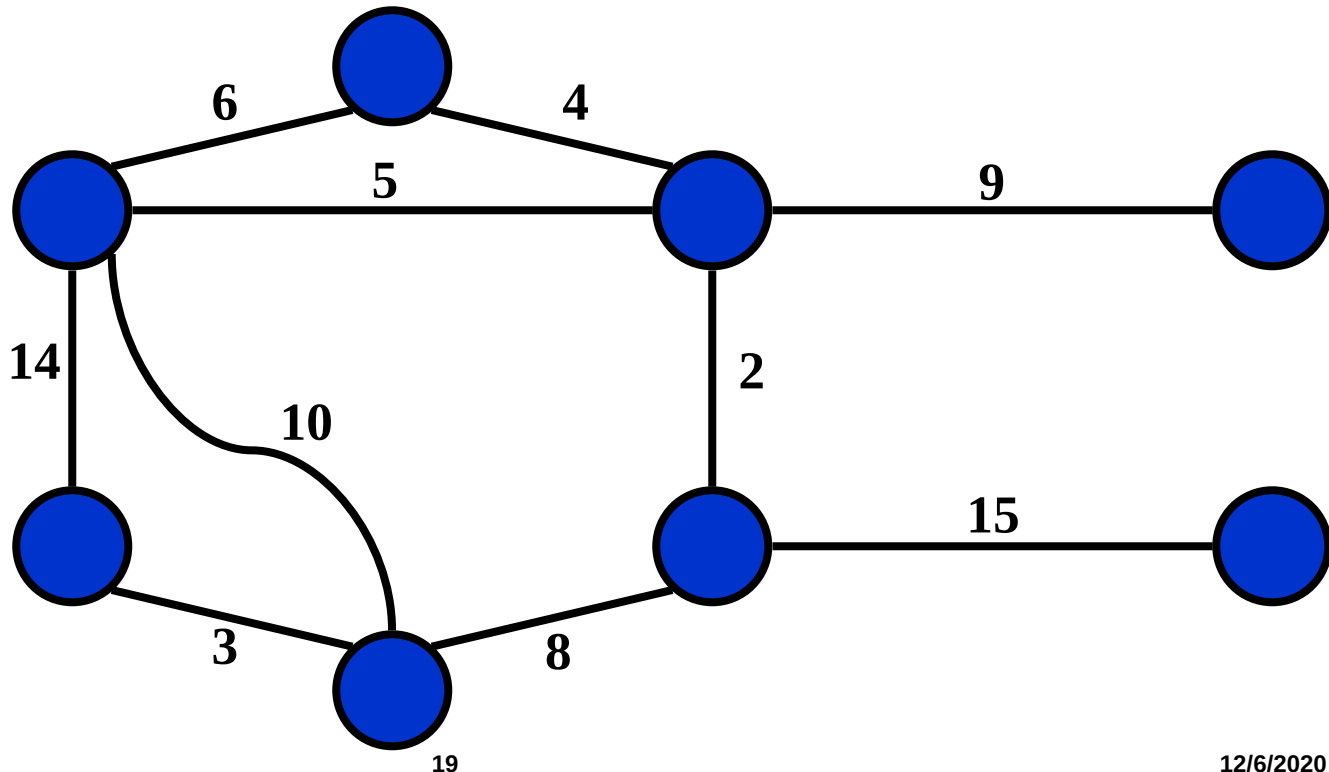
Minimum Spanning Tree

- Problem: given a connected, undirected, weighted graph:



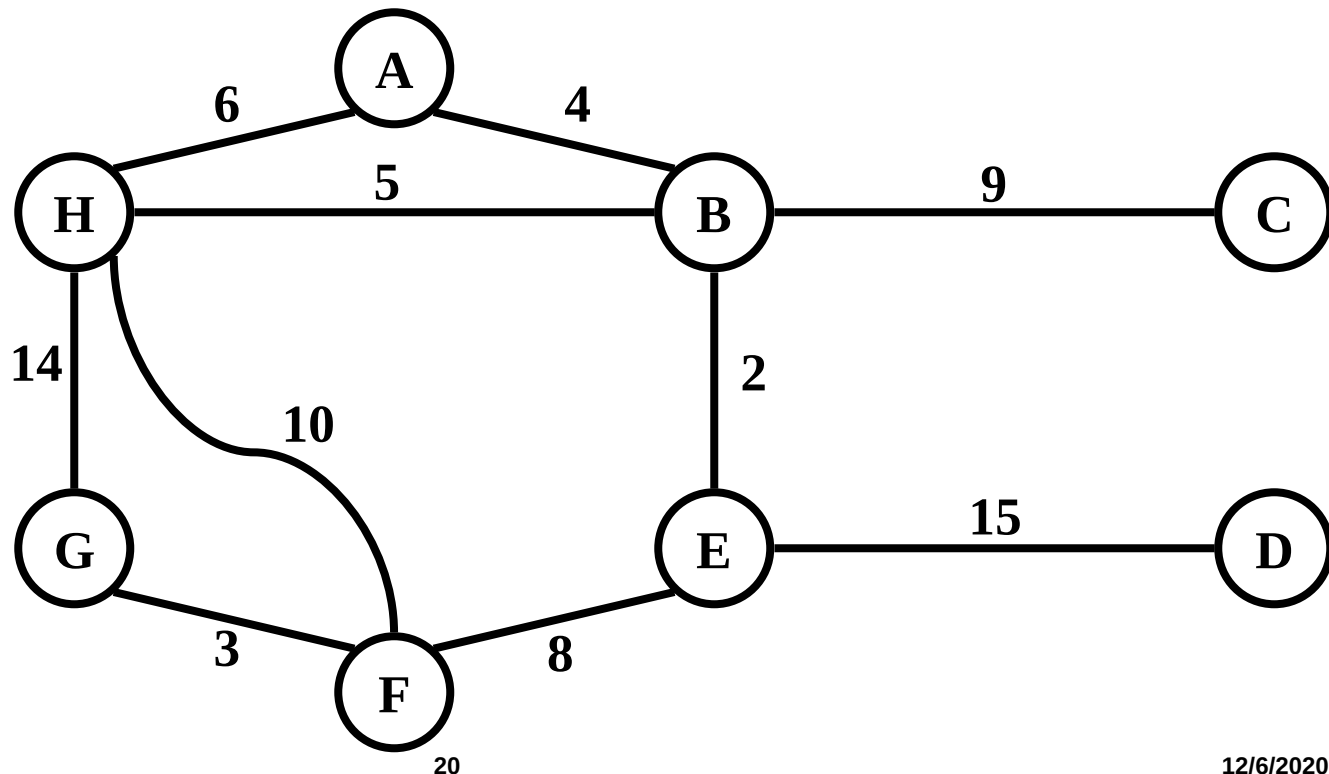
Minimum Spanning Tree

- Problem: given a connected, undirected, weighted graph, find a *spanning tree* using edges that minimize the total weight



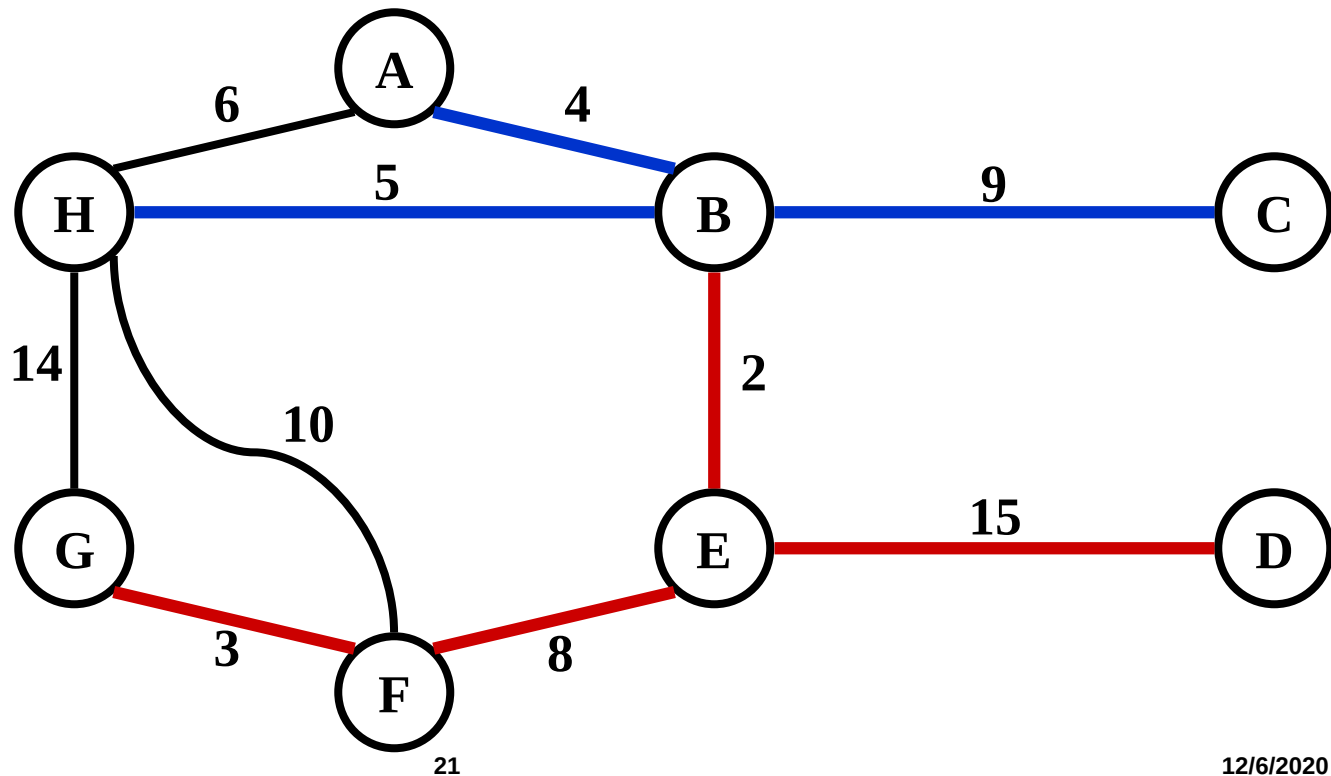
Minimum Spanning Tree

- Which edges form the minimum spanning tree (MST) of the below graph?



Minimum Spanning Tree

- Answer:



Minimum Spanning Tree

- MSTs satisfy the *optimal substructure* property: an optimal tree is composed of optimal subtrees
 - Let T be an MST of G with an edge (u,v) in the middle
 - Removing (u,v) partitions T into two trees T_1 and T_2
 - Claim: T_1 is an MST of $G_1 = (V_1, E_1)$, and T_2 is an MST of $G_2 = (V_2, E_2)$ (*Do V_1 and V_2 share vertices? Why?*)
 - Proof: $w(T) = w(u,v) + w(T_1) + w(T_2)$
(There can't be a better tree than T_1 or T_2 , or T would be suboptimal)

Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

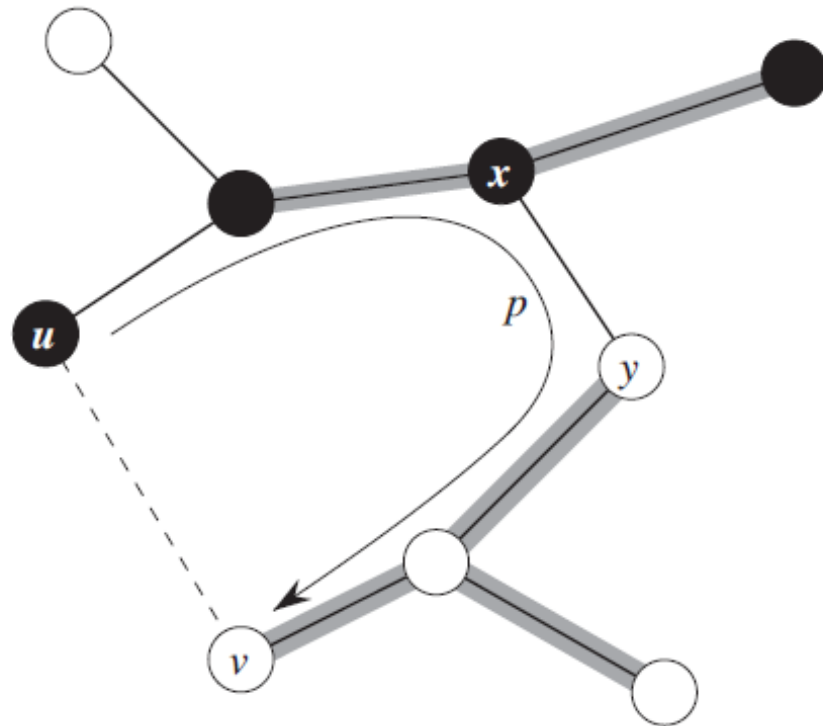
Initialization: After line 1, the set A trivially satisfies the loop invariant.

Maintenance: The loop in lines 2–4 maintains the invariant by adding only safe edges.

Termination: All edges added to A are in a minimum spanning tree, and so the set A returned in line 5 must be a minimum spanning tree.

Minimum Spanning Tree

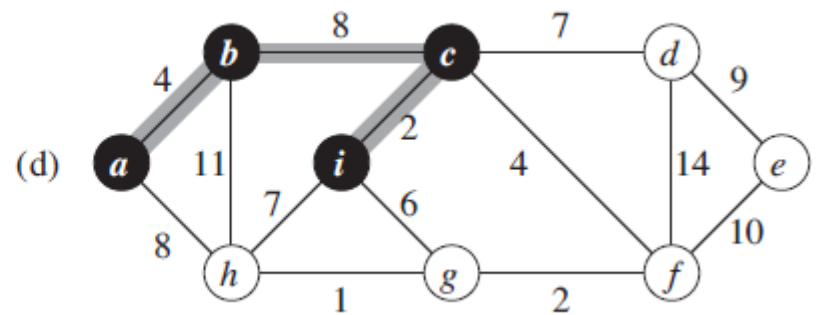
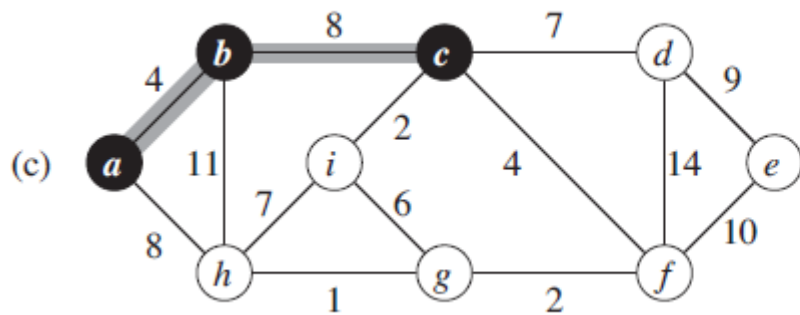
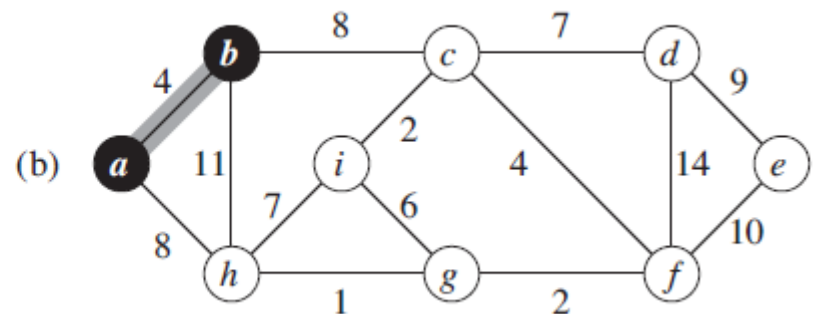
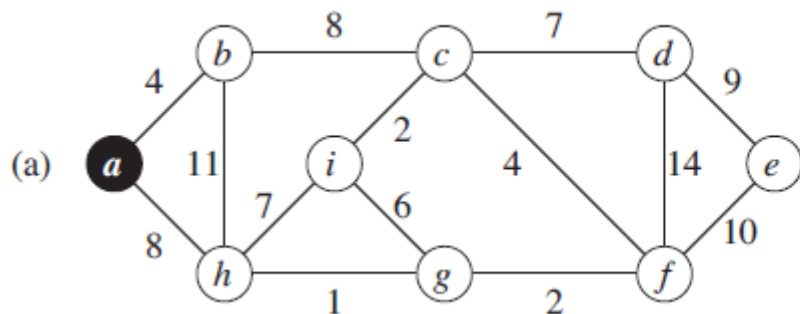
- Thm 23.1 :
 - Let T be MST of G , and let $A \subseteq T$ be subtree of T
 - Let (u,v) be min-weight edge connecting A to $V-A$
 - Then $(u,v) \in T$
- Proof:
By exchange..

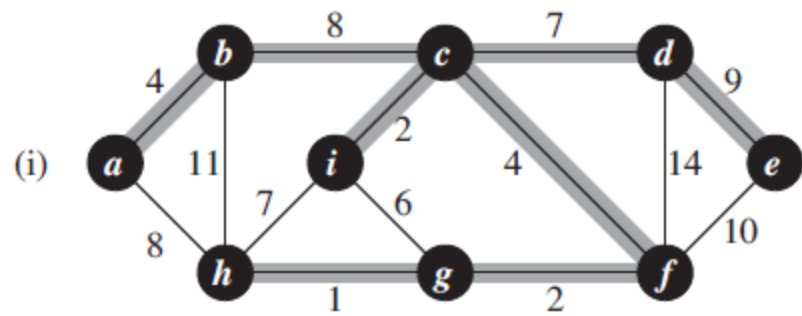
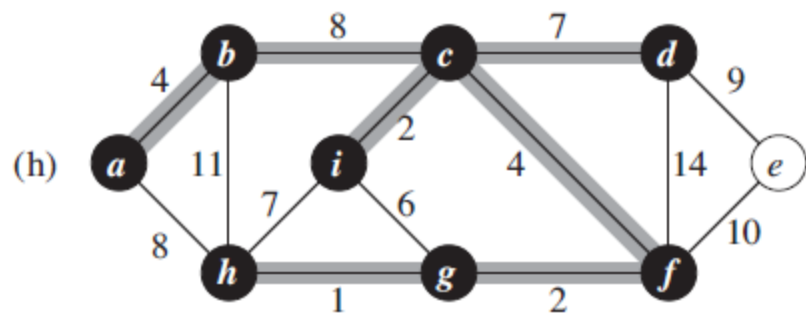
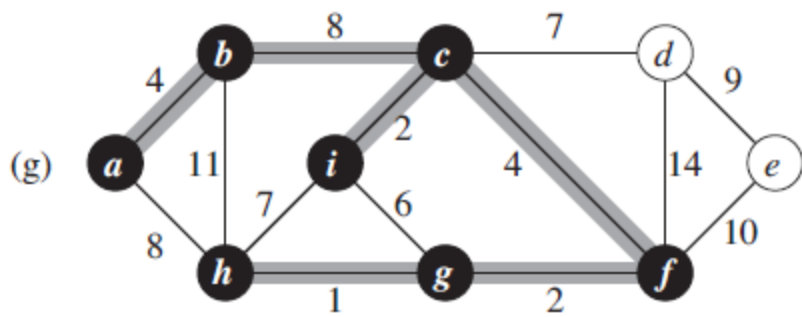
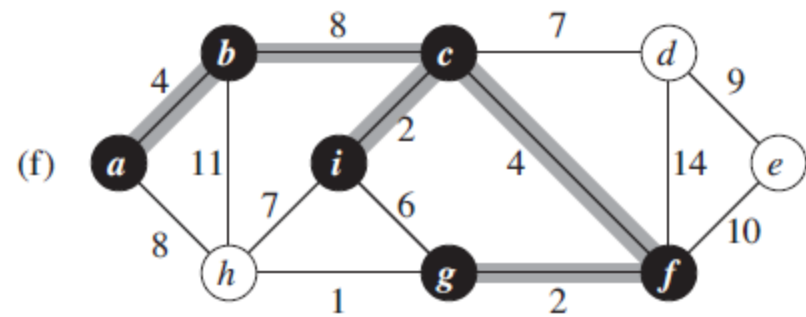
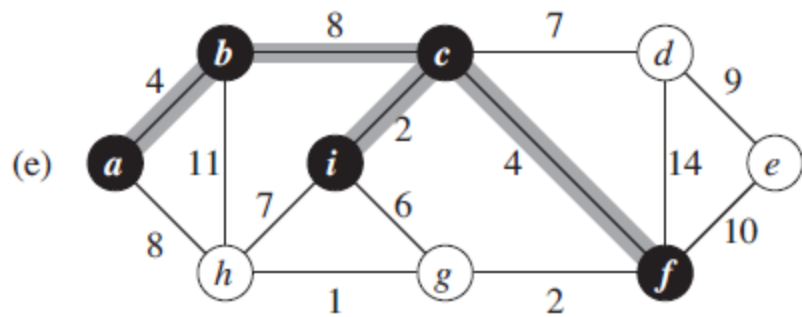


Prim's Algorithm

```
MST-Prim( $G, w, r$ )
   $Q = V[G];$ 
  for each  $u \in Q$ 
     $key[u] = \infty;$ 
   $key[r] = 0;$ 
   $p[r] = \text{NULL};$ 
  while ( $Q$  not empty)
     $u = \text{ExtractMin}(Q);$ 
    for each  $v \in \text{Adj}[u]$ 
      if ( $v \in Q$  and  $w(u, v) < key[v]$ )
         $p[v] = u;$ 
         $key[v] = w(u, v);$ 
```

Prim's Algorithm





Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

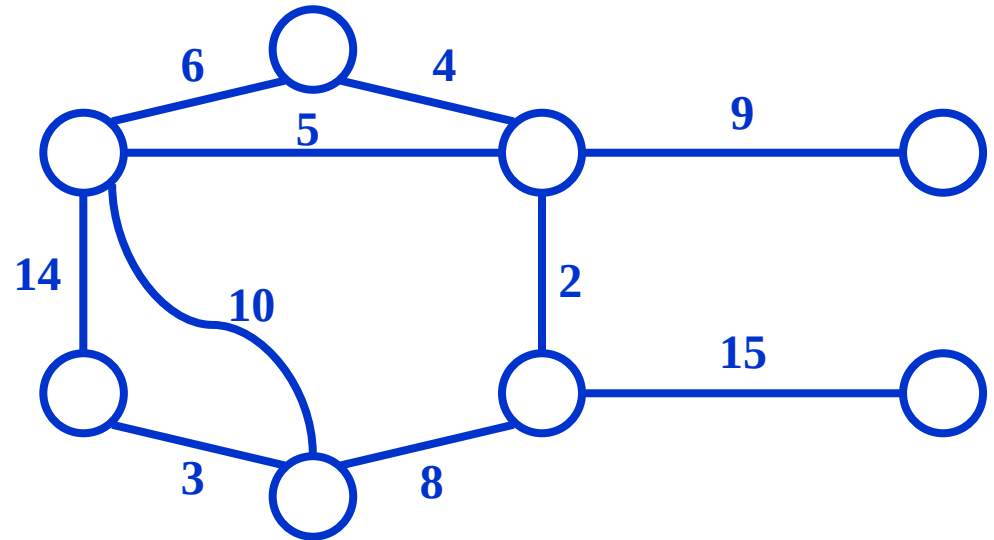
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Run on example graph

Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

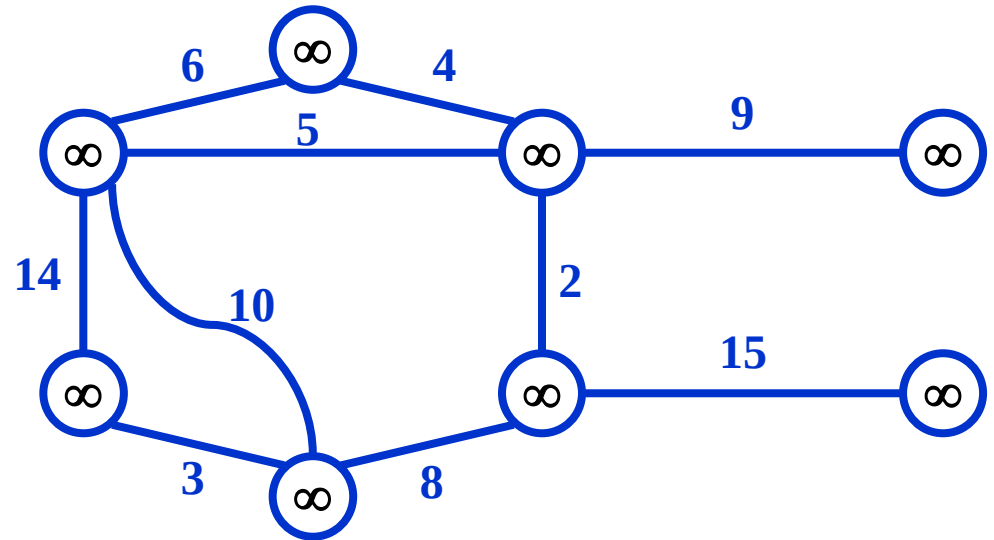
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Run on example graph

Prim's Algorithm

MST-Prim(G, w, r)

$Q = V[G];$

for each $u \in Q$

$\text{key}[u] = \infty;$

$\text{key}[r] = 0;$

$p[r] = \text{NULL};$

while (Q not empty)

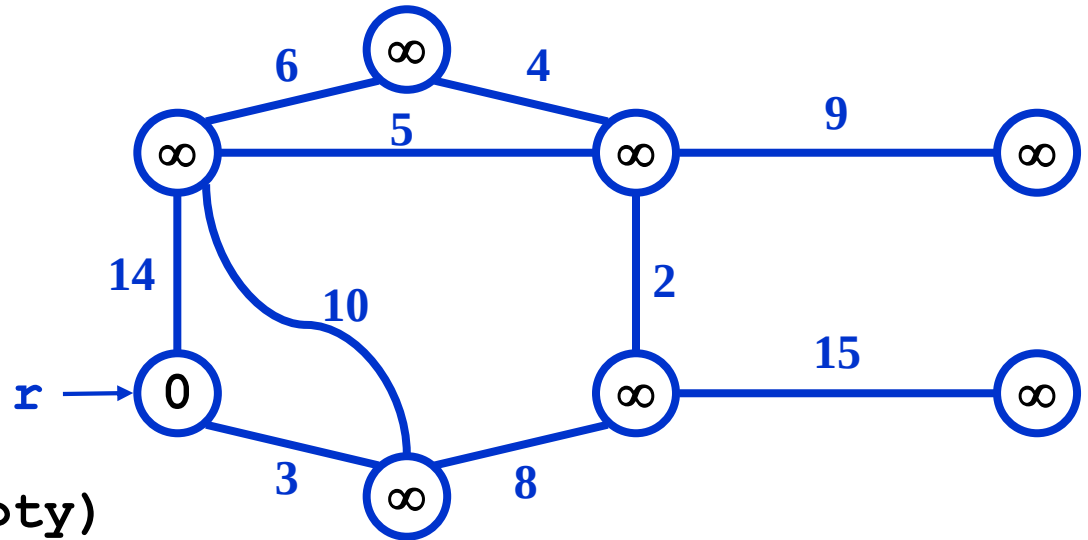
$u = \text{ExtractMin}(Q);$

 for each $v \in \text{Adj}[u]$

 if ($v \in Q$ and $w(u, v) < \text{key}[v]$)

$p[v] = u;$

$\text{key}[v] = w(u, v);$



Pick a start vertex r

Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

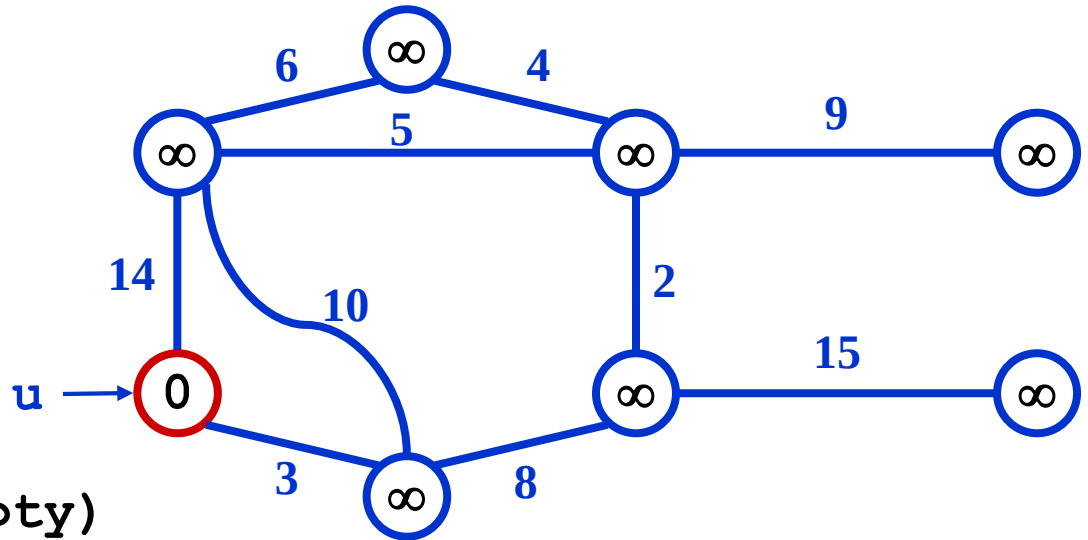
`$u = \text{ExtractMin}(Q);$  Red vertices have been removed from  $Q$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

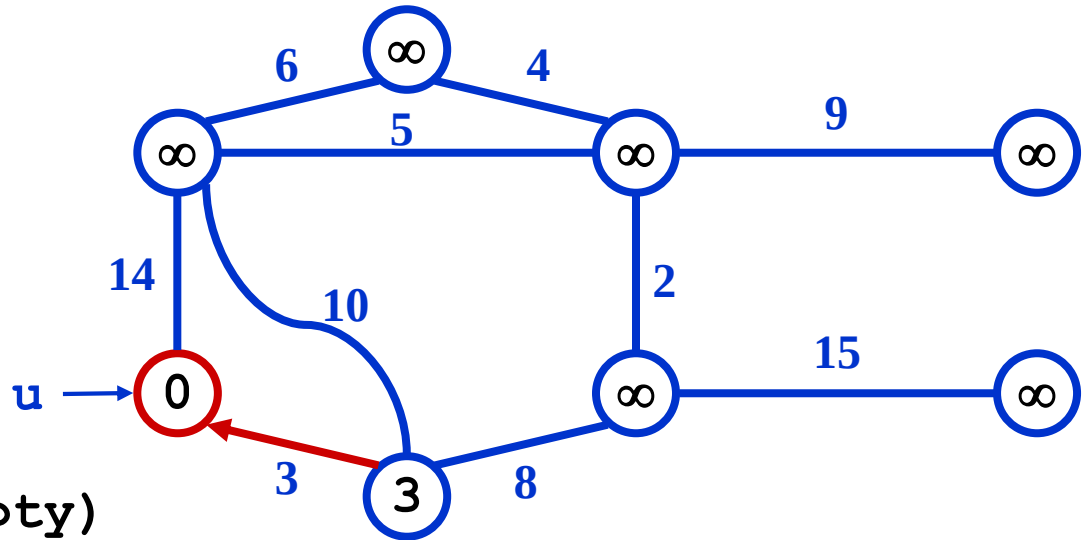
`$u = \text{ExtractMin}(Q);$     Red arrows indicate parent pointers`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

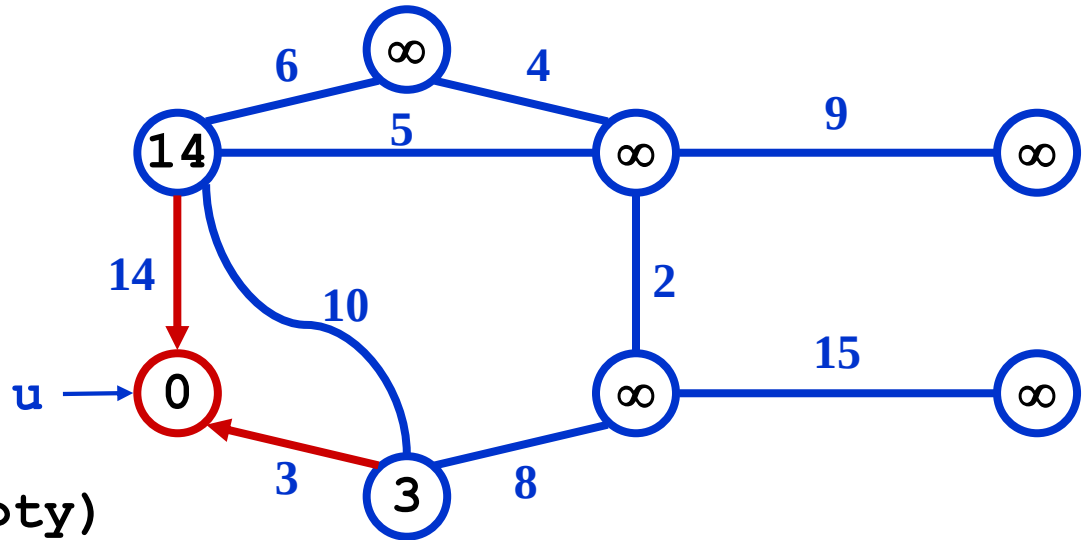
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

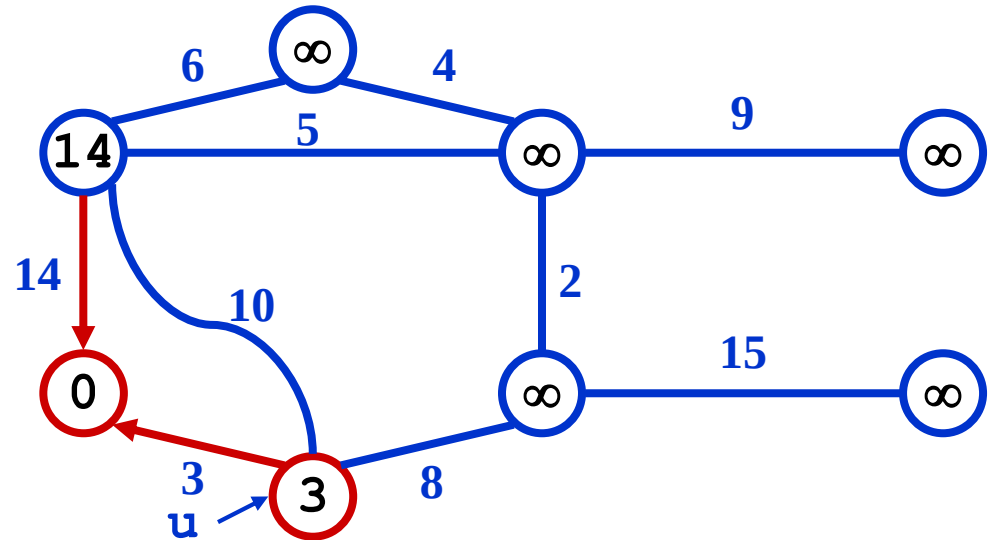
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

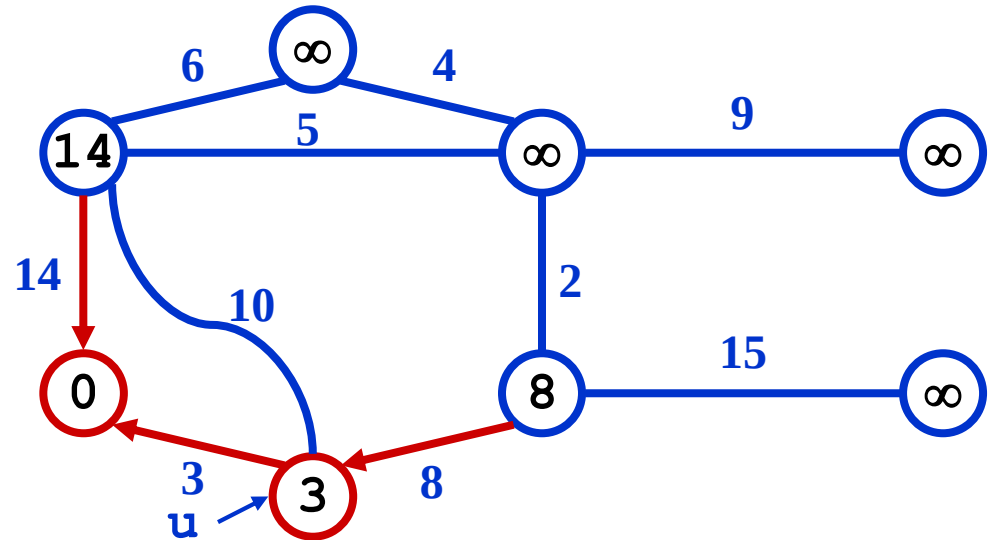
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

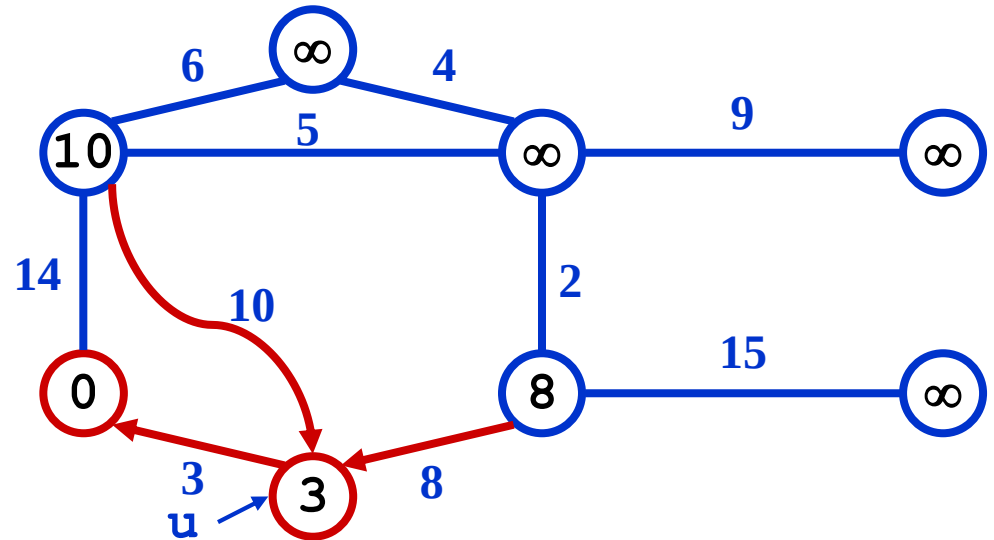
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

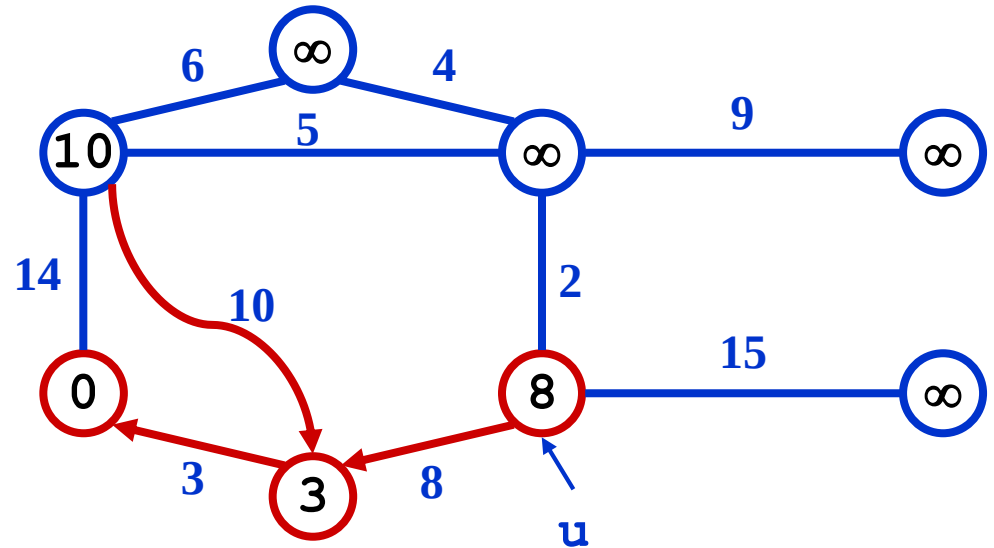
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

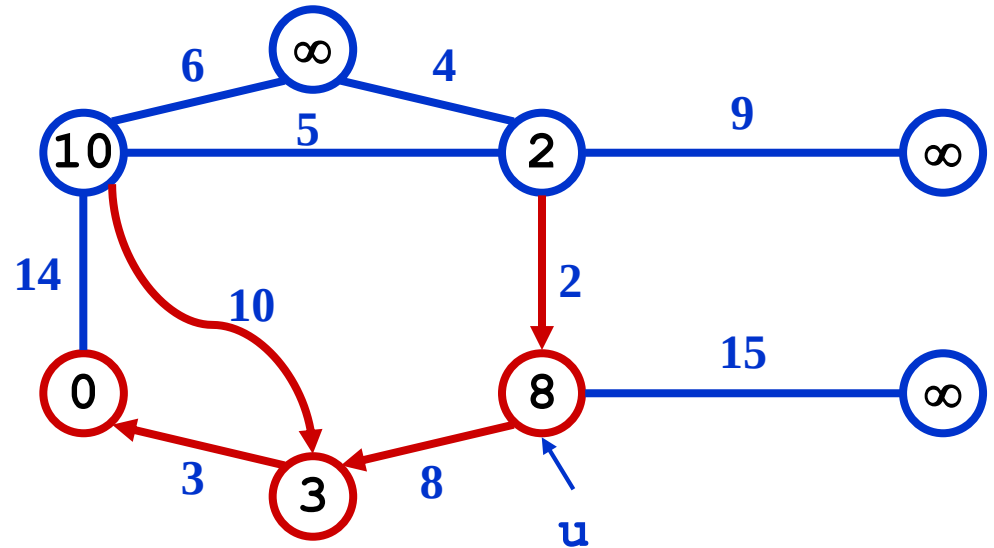
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

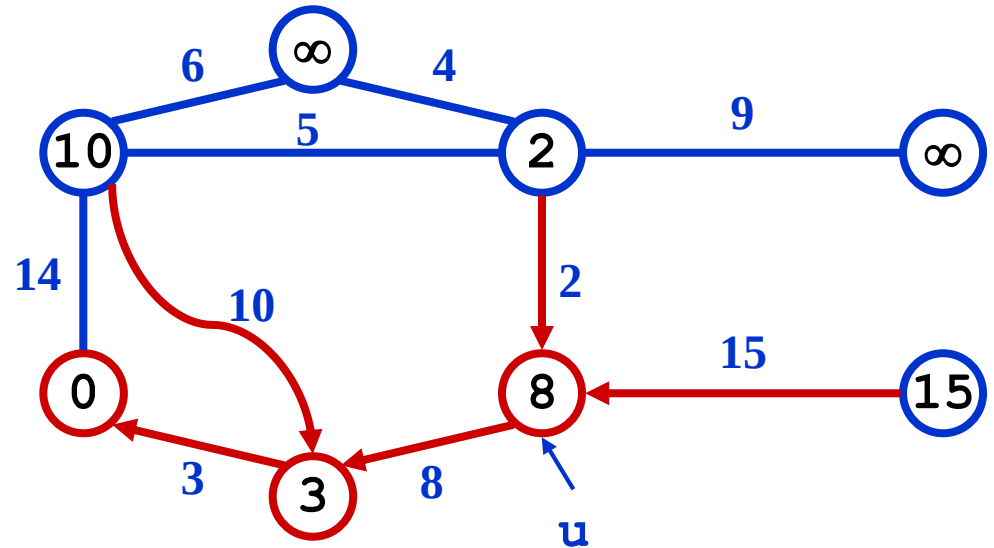
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

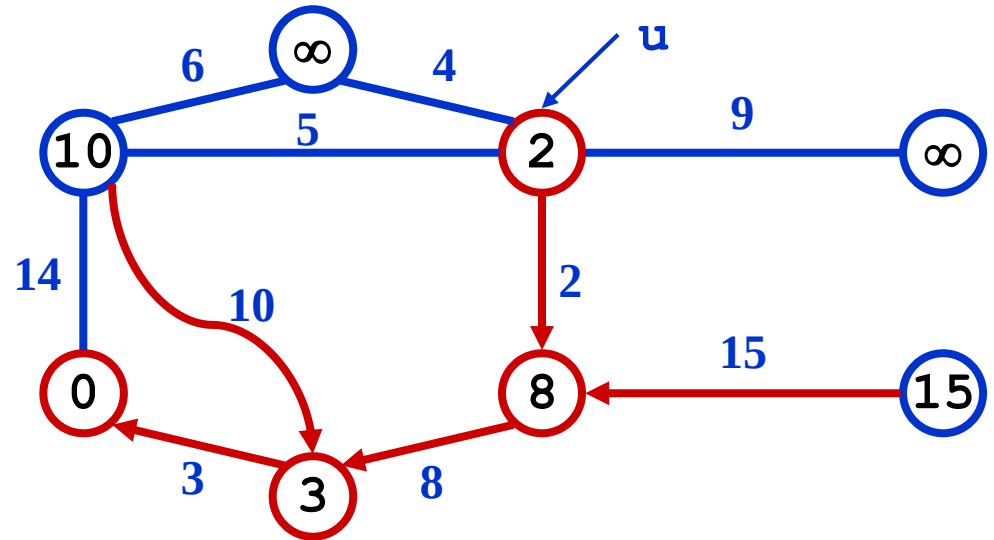
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

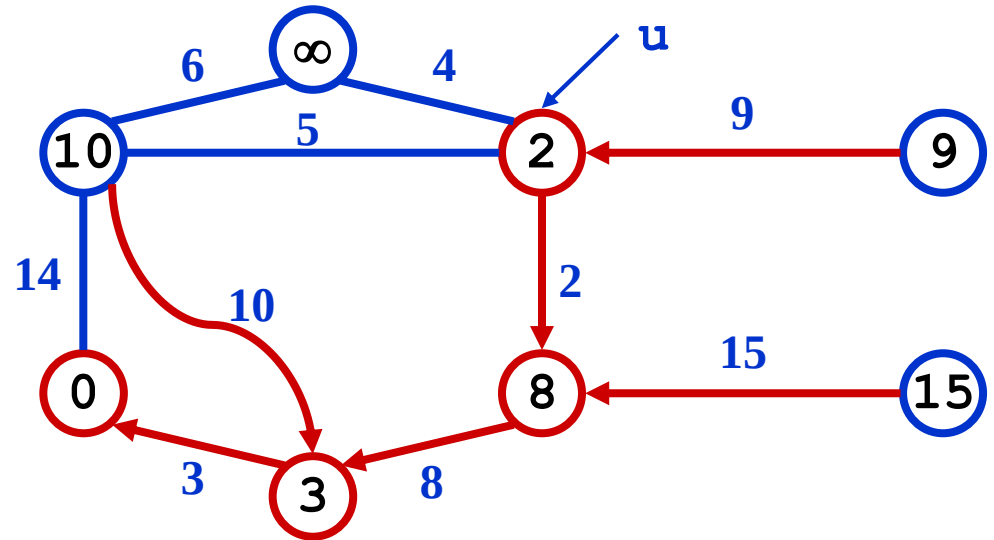
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

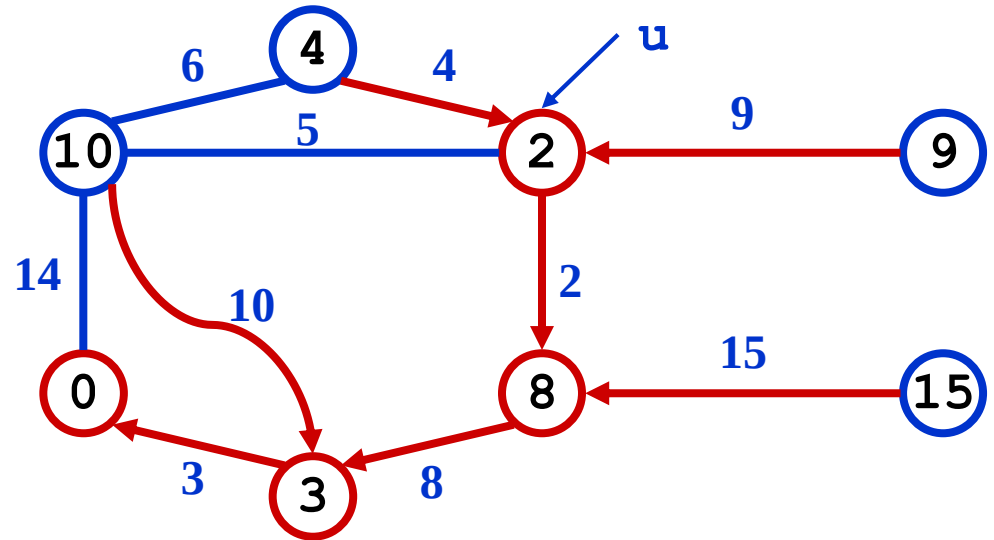
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

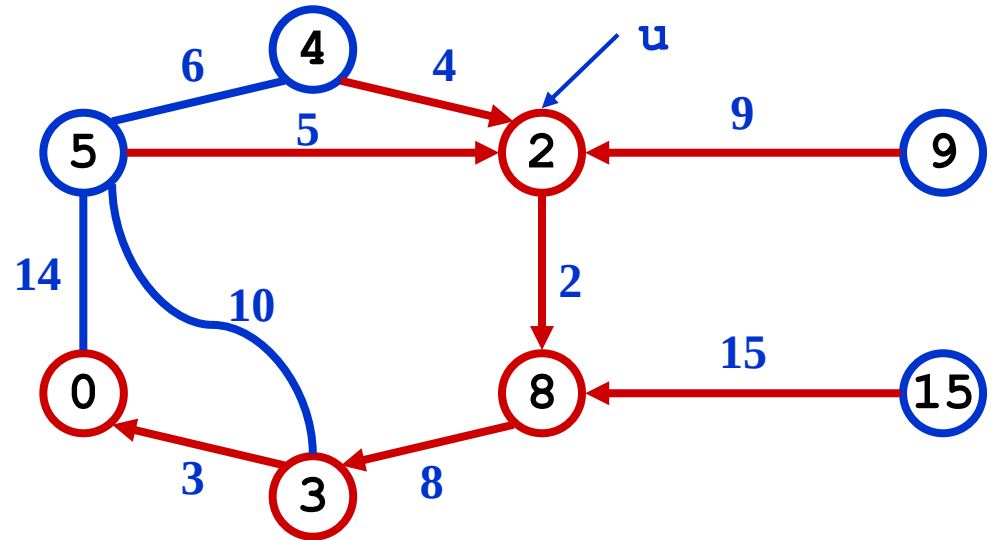
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

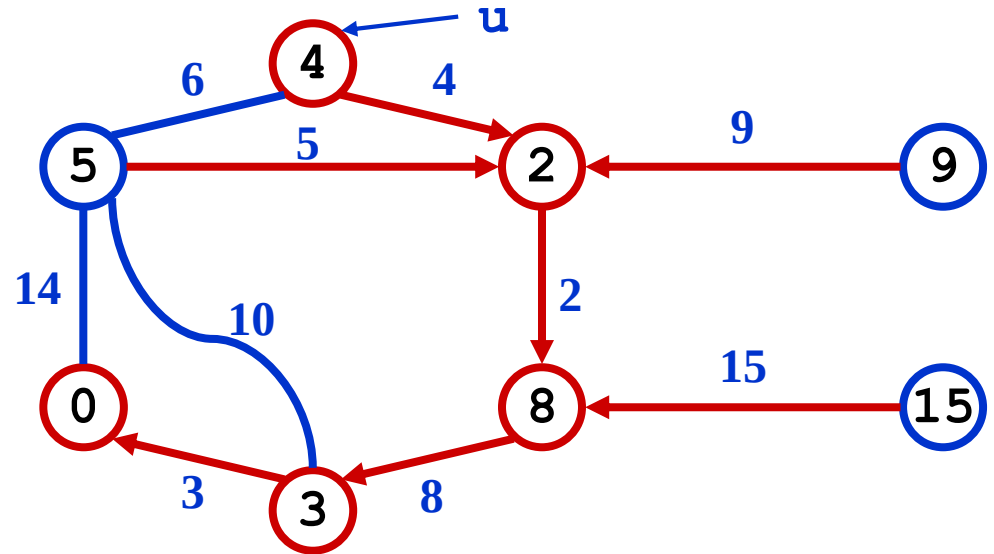
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

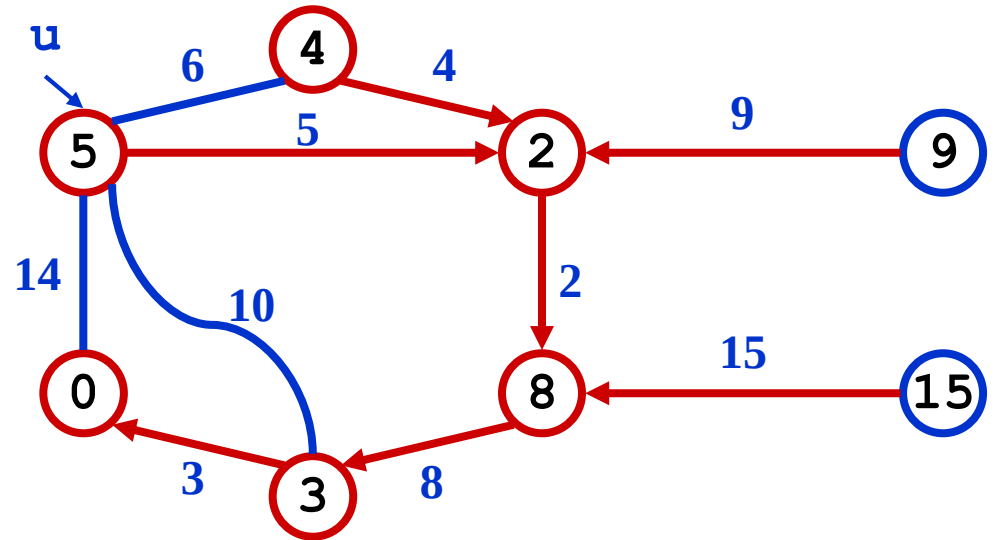
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $\text{key}[u] = \infty;$ 
```

```
   $\text{key}[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

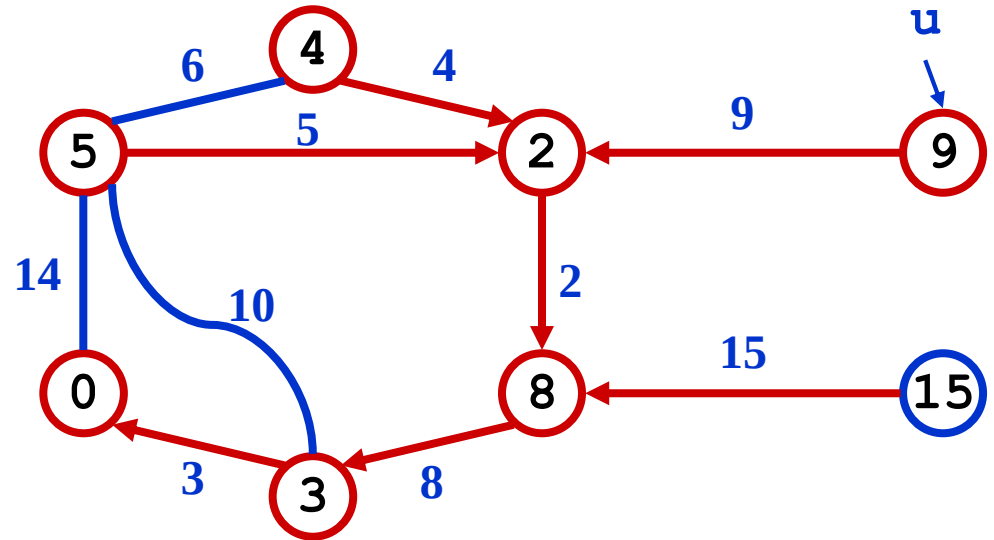
```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $\text{key}[v] = w(u, v);$ 
```



Prim's Algorithm

`MST-Prim( $G, w, r$ )`

`$Q = V[G];$`

`for each  $u \in Q$`

`$\text{key}[u] = \infty;$`

`$\text{key}[r] = 0;$`

`$p[r] = \text{NULL};$`

`while ( $Q$  not empty)`

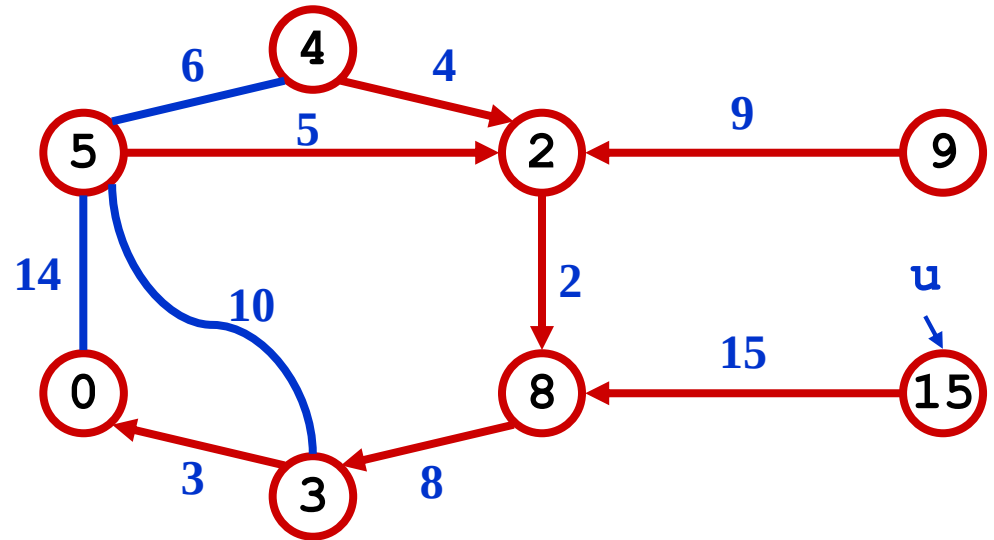
`$u = \text{ExtractMin}(Q);$`

`for each  $v \in \text{Adj}[u]$`

`if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )`

`$p[v] = u;$`

`$\text{key}[v] = w(u, v);$`



Analysis: Prim's Algorithm

```
MST-Prim( $G, w, r$ )
```

```
   $Q = V[G];$ 
```

```
  for each  $u \in Q$ 
```

```
     $key[u] = \infty;$ 
```

```
   $key[r] = 0;$ 
```

```
   $p[r] = \text{NULL};$ 
```

```
  while ( $Q$  not empty)
```

```
     $u = \text{ExtractMin}(Q);$ 
```

```
    for each  $v \in \text{Adj}[u]$ 
```

```
      if ( $v \in Q$  and  $w(u, v) < key[v]$ )
```

```
         $p[v] = u;$ 
```

```
         $key[v] = w(u, v);$ 
```

What is the hidden cost in this code?

Analysis: Prim's Algorithm

```
MST-Prim( $G, w, r$ )
   $Q = V[G];$ 
  for each  $u \in Q$ 
     $key[u] = \infty;$ 
   $key[r] = 0;$ 
   $p[r] = \text{NULL};$ 
  while ( $Q$  not empty)
     $u = \text{ExtractMin}(Q);$ 
    for each  $v \in \text{Adj}[u]$ 
      if ( $v \in Q$  and  $w(u, v) < key[v]$ )
         $p[v] = u;$ 
        DecreaseKey( $v, w(u, v)$ );
```

Analysis: Prim's Algorithm

MST-Prim(G, w, r)

$Q = V[G];$

for each $u \in Q$

$\text{key}[u] = \infty;$ *How often is ExtractMin() called?*

$\text{key}[r] = 0;$

How often is DecreaseKey() called?

$p[r] = \text{NULL};$

while (Q not empty)

$u = \text{ExtractMin}(Q);$

 for each $v \in \text{Adj}[u]$

 if ($v \in Q$ and $w(u, v) < \text{key}[v]$)

$p[v] = u;$

 DecreaseKey($v, w(u, v)$);

Analysis: Prim's Algorithm

MST-Prim(G, w, r)

$Q = V[G];$

for each $u \in Q$

$key[u] = \infty;$

$key[r] = 0;$

$p[r] = \text{NULL};$

while (Q not empty)

$u = \text{ExtractMin}(Q);$

 for each $v \in \text{Adj}[u]$

 if ($v \in Q$ and $w(u, v) < key[v]$)

$p[v] = u;$

$key[v] = w(u, v);$

What will be the running time?

A: Depends on queue

binary heap: $O(E \lg V)$

Fibonacci heap: $O(V \lg V + E)$

Kruskal's Algorithm

- Starts with each vertex in its own component.
- **Repeatedly merges two components** into one by choosing a light edge that connects them (i.e., a light edge crossing the cut between them).
- Scans the set of edges in monotonically increasing order by weight.
- Uses a **disjoint-set data structure** to determine whether an edge connects vertices in different components.

Disjoint Set Data Structure

- Want to maintain a collection $S = \{S_1, \dots, S_k\}$ of **disjoint dynamic sets**.
- Each set has a **representative member**.
- **Operations:**
 - **Make-Set(x)**: Make new singleton set containing object x (x is representative).
 - **Union(x, y)**: Unite sets containing x and y into a new set that is a union of the two sets. (Representative is any member of the union.)
 - **Find-Set(x)**: Returns pointer to representative of set containing x .
- **Complexity:** $O(m \alpha(n))$
 - n = no. of Make-Set operations, m = total no. of operations.
 - **Note:** $m \geq n$.
 - Suffices to know $\alpha(n) = O(\lg^* n) = O(\lg n)$.
 $\lg^*(n) = \min\{i \geq 0 : \lg^{(i)} n \leq 1\}$

Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Kruskal's Algorithm

MST-KRUSKAL(G, w)

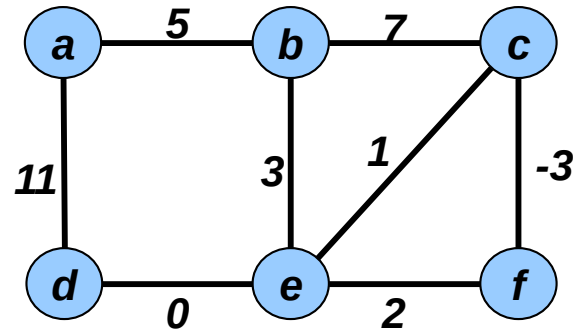
```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

- Initialization -- $O(V)$.
- Sort Edges -- $O(E \lg E)$.

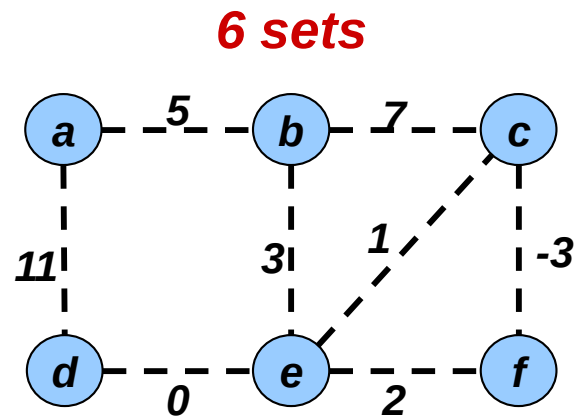
- Set Operations --
 $O((V+E)\alpha(V))$
- Total is $O(E \lg E)$.

- $E = O(V^2)$
- Therefore, $\lg E = O(\lg V)$
- Hence, complexity is
 $O(E \lg V)$.

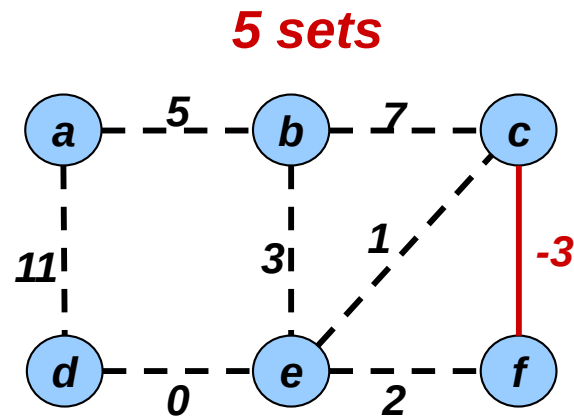
Example of Kruskal's Algorithm



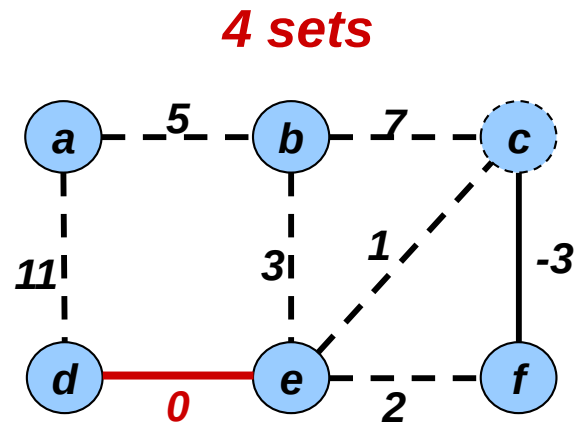
Example of Kruskal's Algorithm



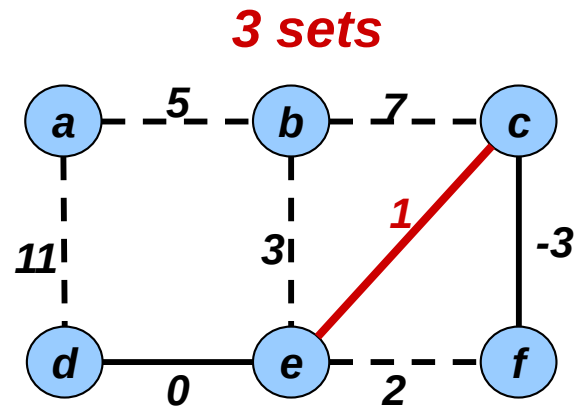
Example of Kruskal's Algorithm



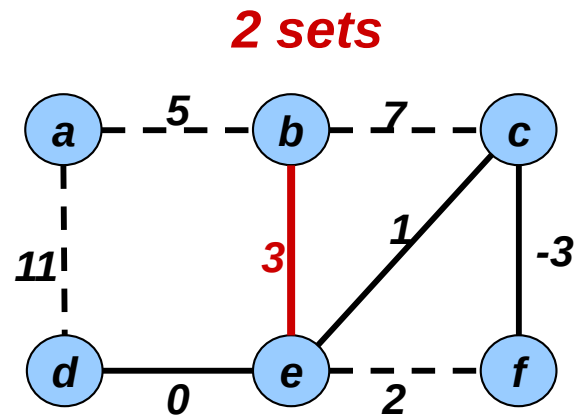
Example of Kruskal's Algorithm



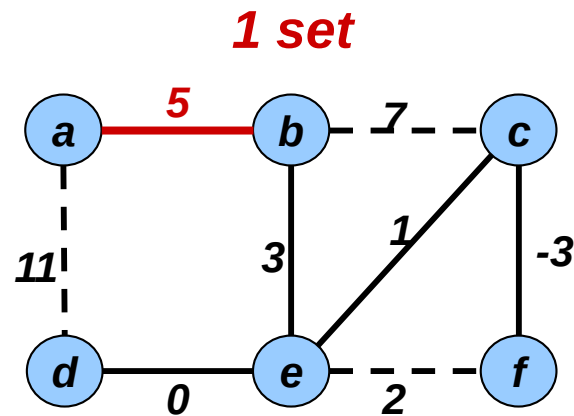
Example of Kruskal's Algorithm



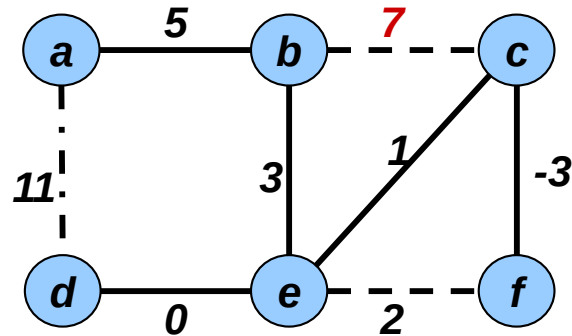
Example of Kruskal's Algorithm



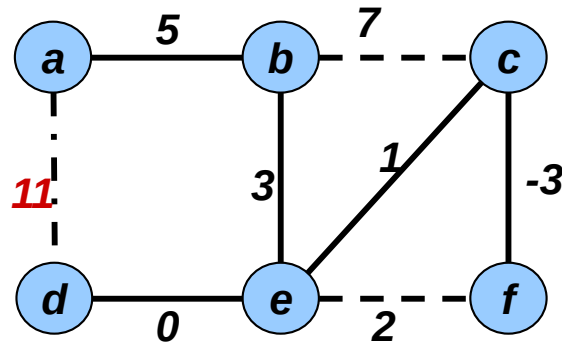
Example of Kruskal's Algorithm



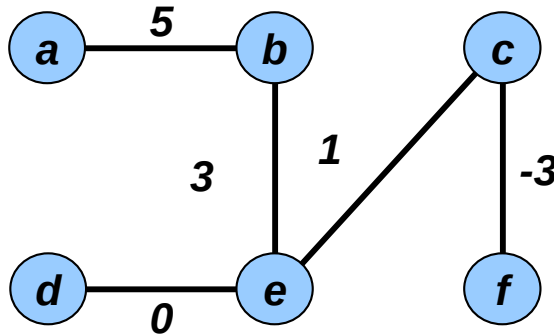
Example of Kruskal's Algorithm

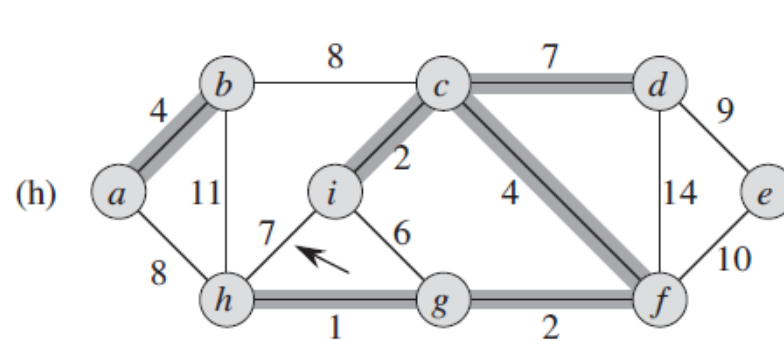
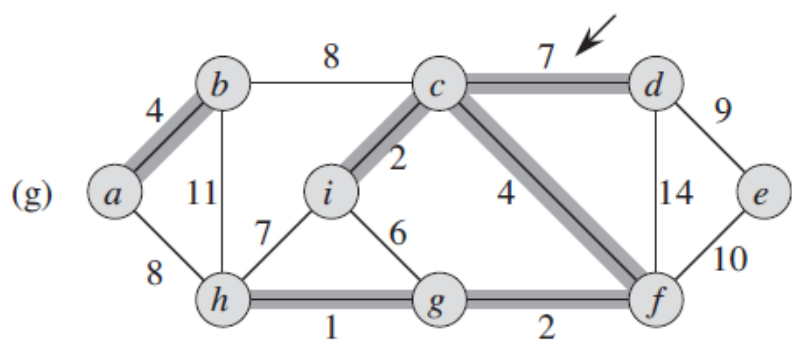
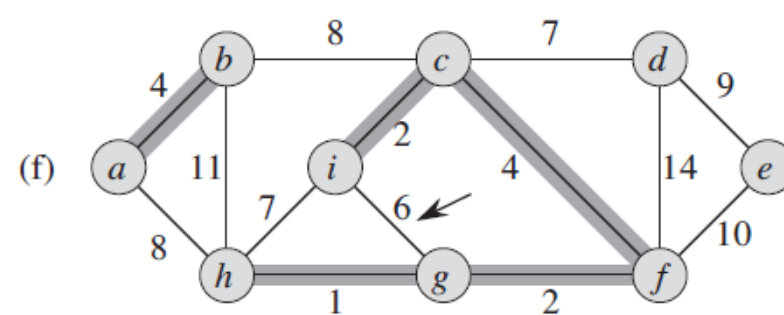
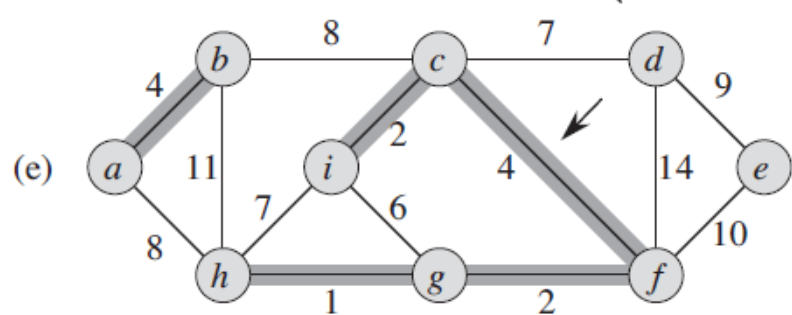
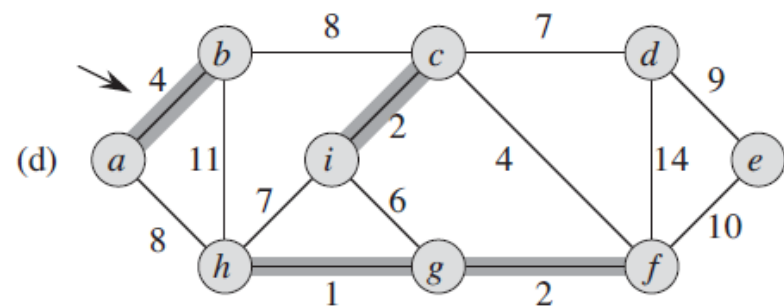
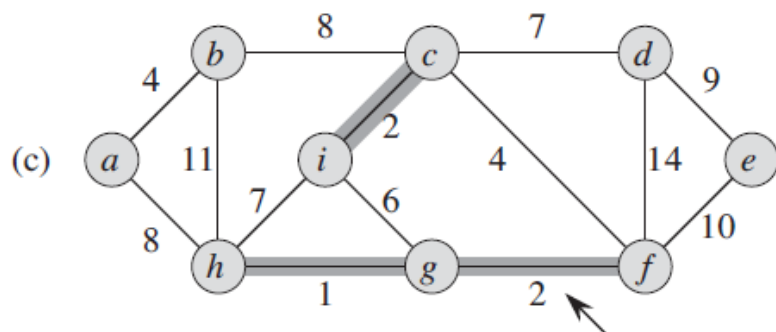
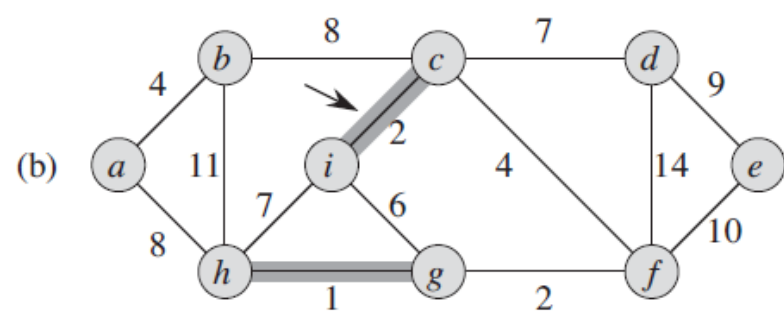
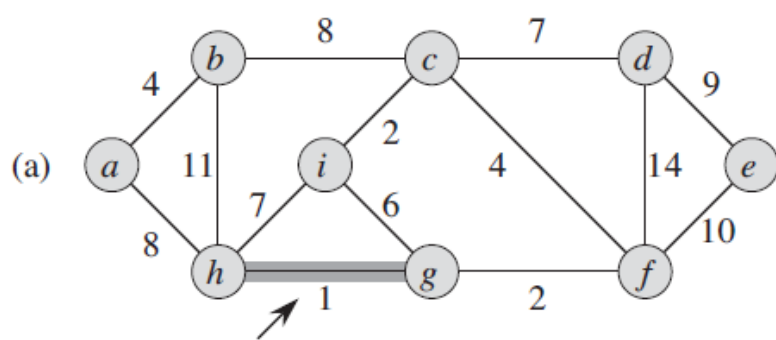


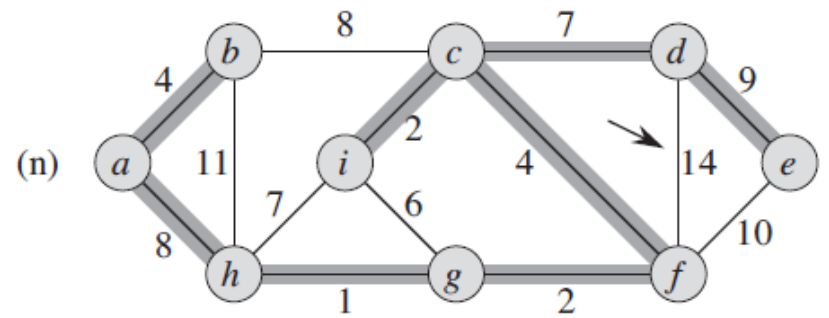
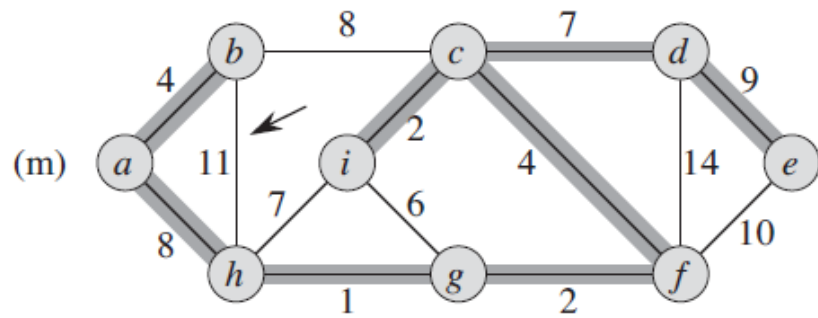
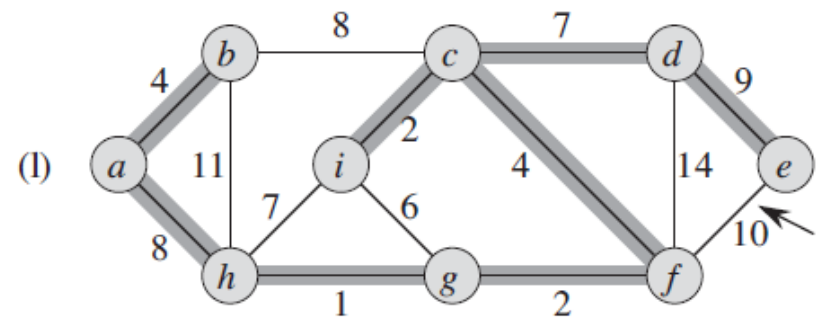
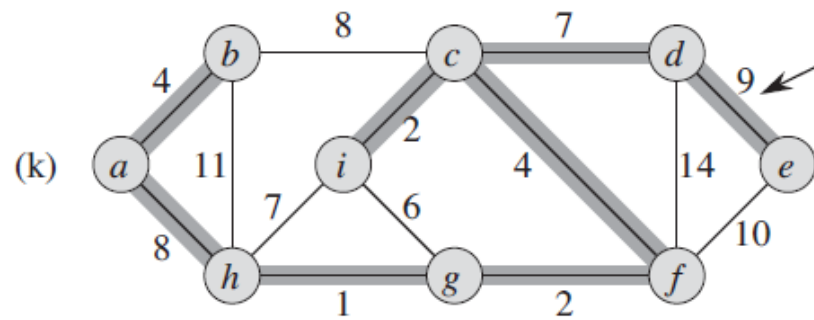
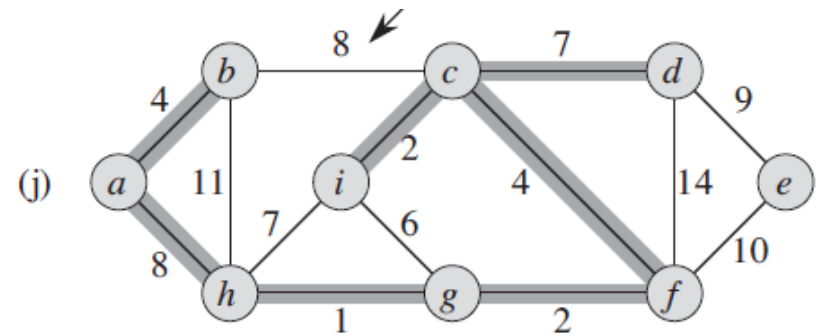
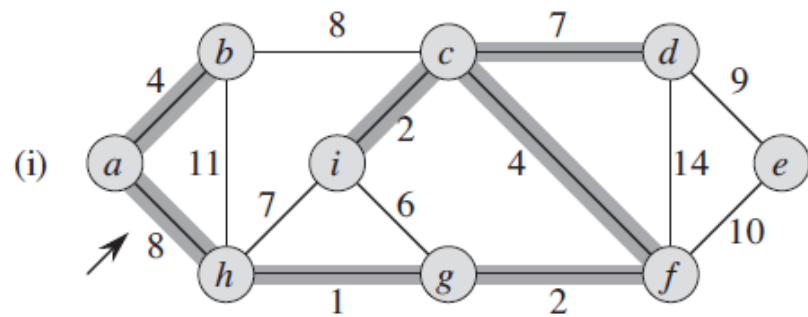
Example of Kruskal's Algorithm

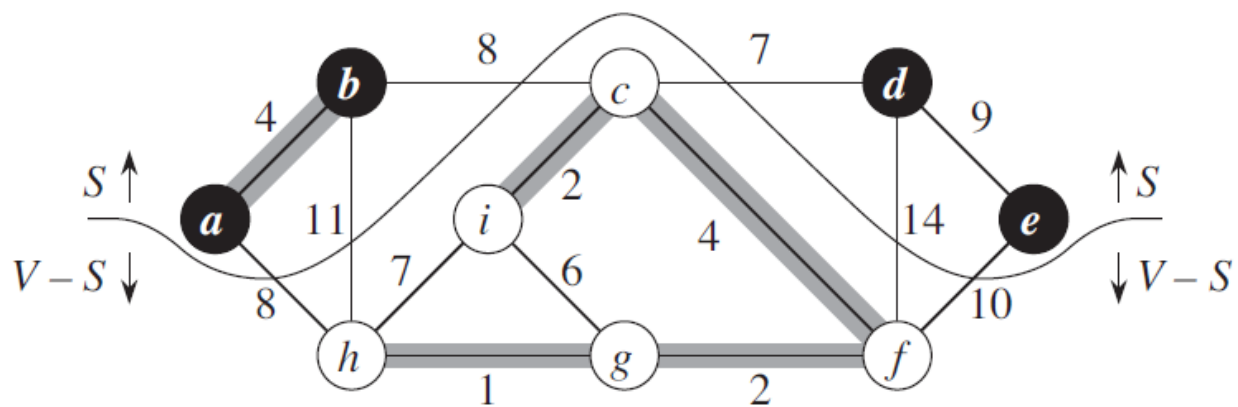


Example of Kruskal's Algorithm

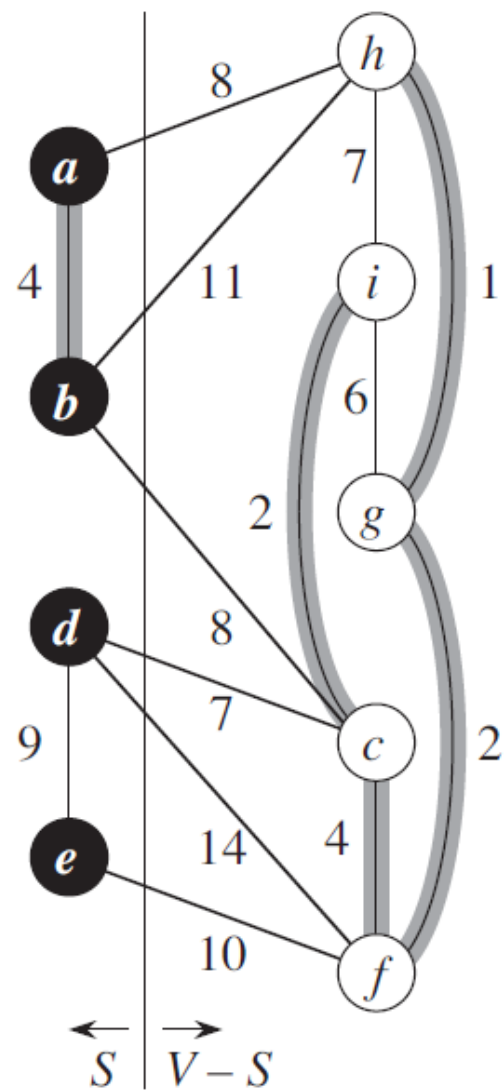








(a)



(b)